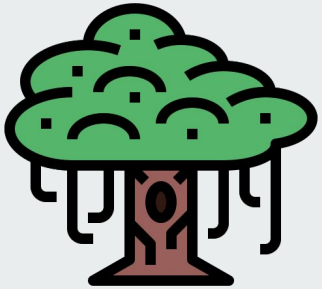
Banyan: Fast Rotating Leader BFT

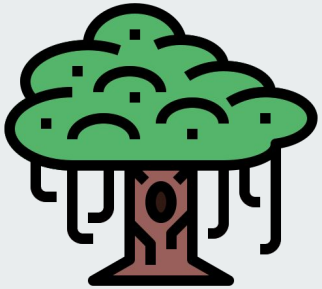
Yann Vonlanthen, Jakub Sliwinski, Massimo Albarello, and Roger Wattenhofer



*ETH Zurich - **Dis**tributed **Co**mputing Group*

Banyan: Fast Rotating Leader BFT

Yann Vonlanthen, Jakub Sliwinski, Massimo Albarello, and Roger Wattenhofer



*ETH Zurich - **Dis**tributed **Co**mputing Group*

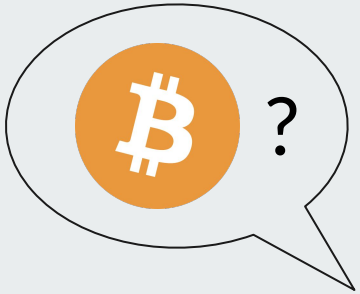
















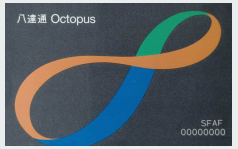










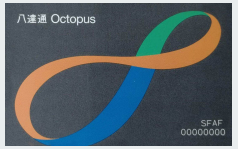

八達通 Octopus
SFAF
00000000










八達通 Octopus
SFAF
00000000

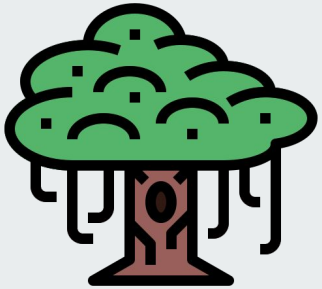






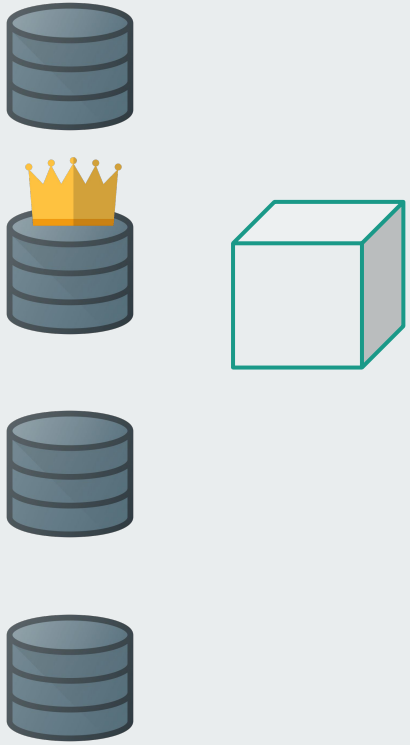
Banyan: Fast Rotating Leader BFT

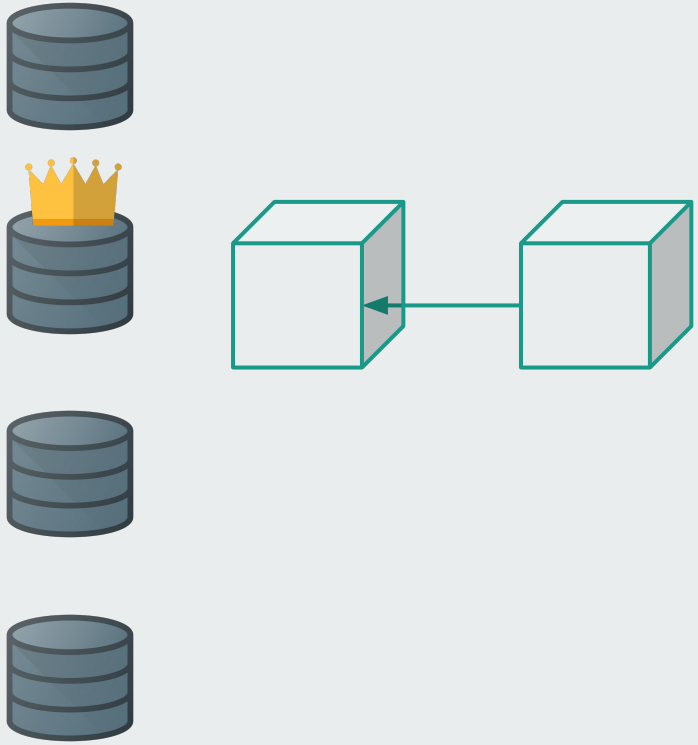
Yann Vonlanthen, Jakub Sliwinski, Massimo Albarello, and Roger Wattenhofer

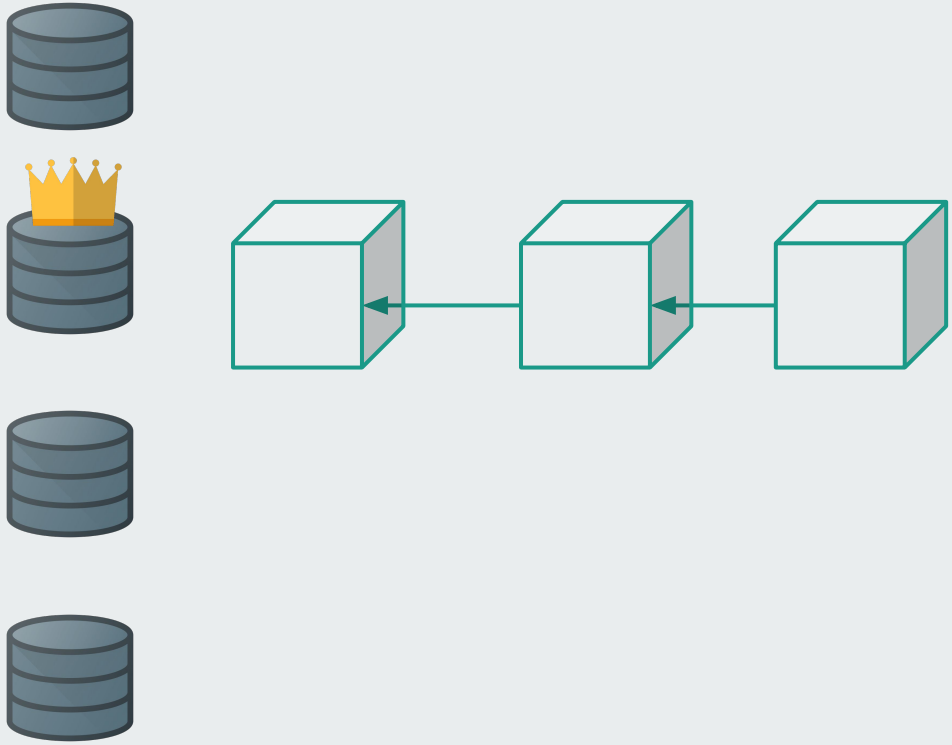


*ETH Zurich - **Dis**tributed **Co**mputing Group*

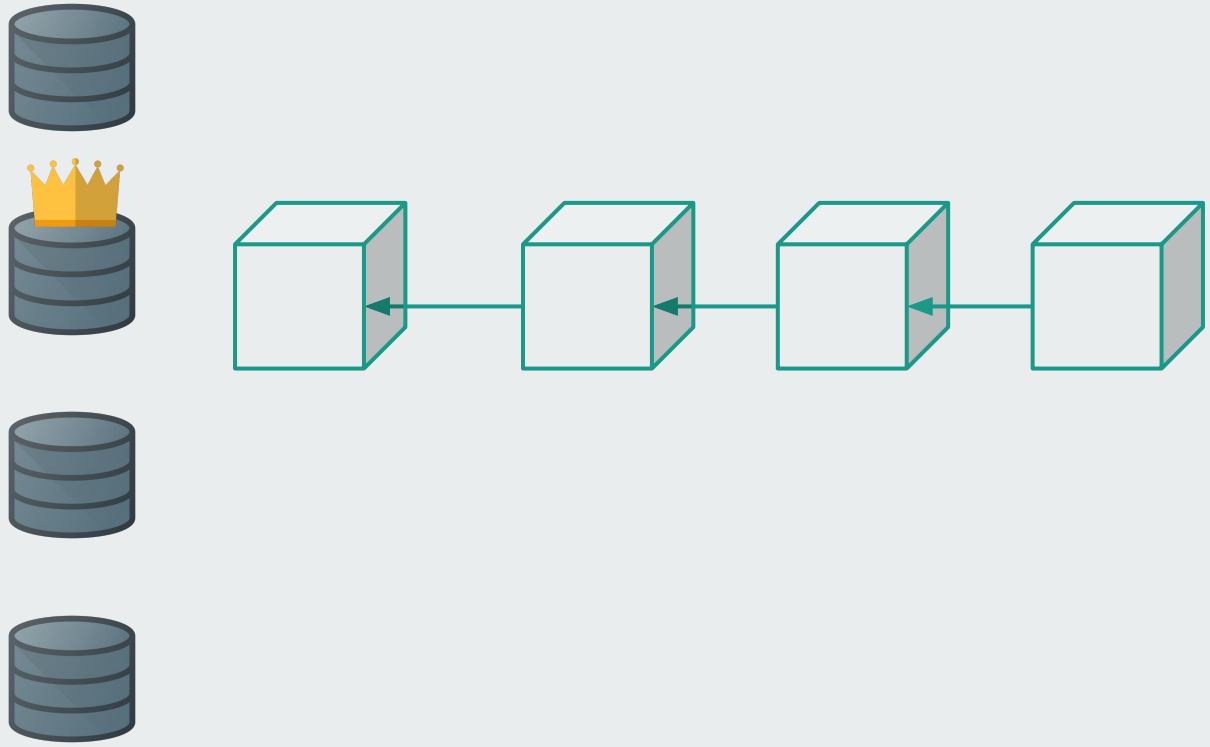


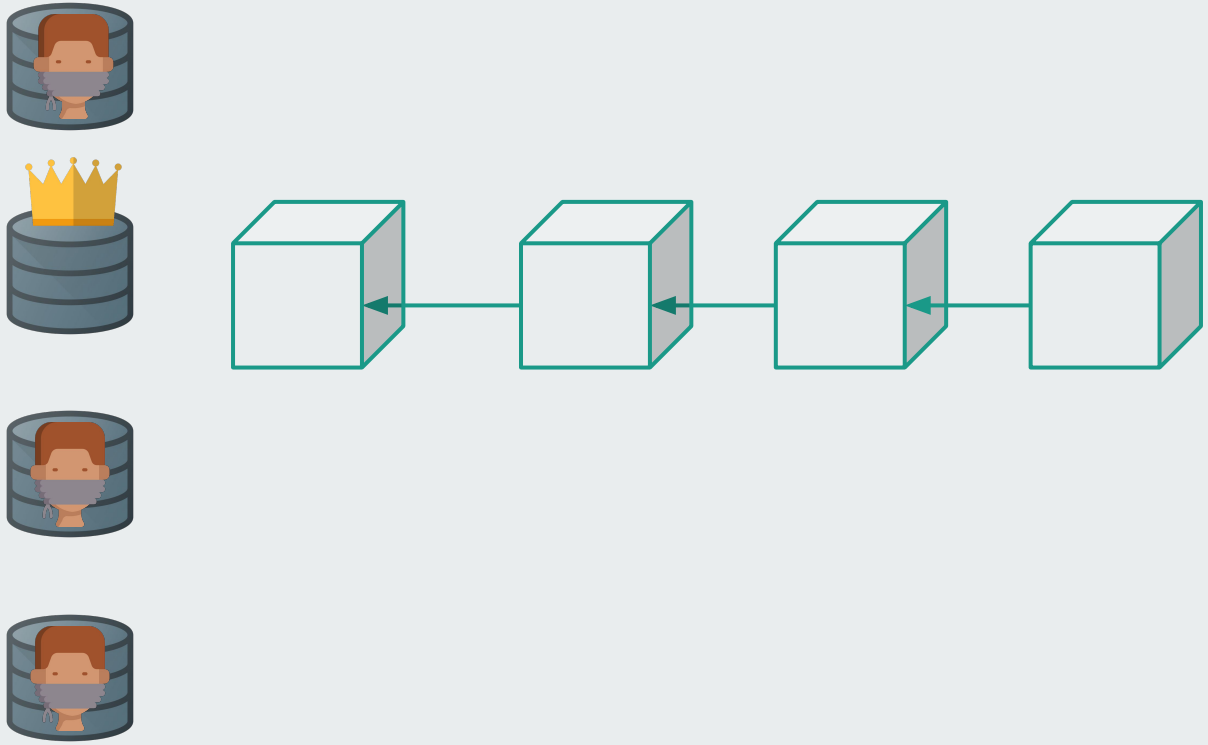


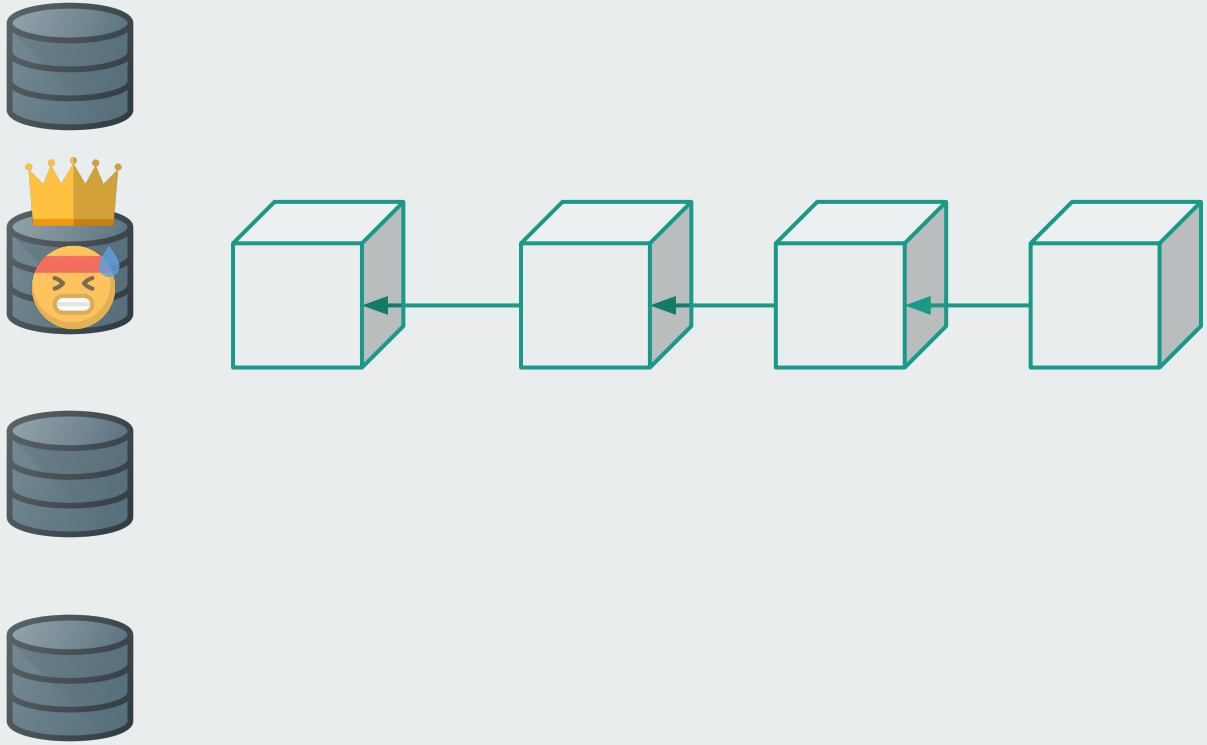


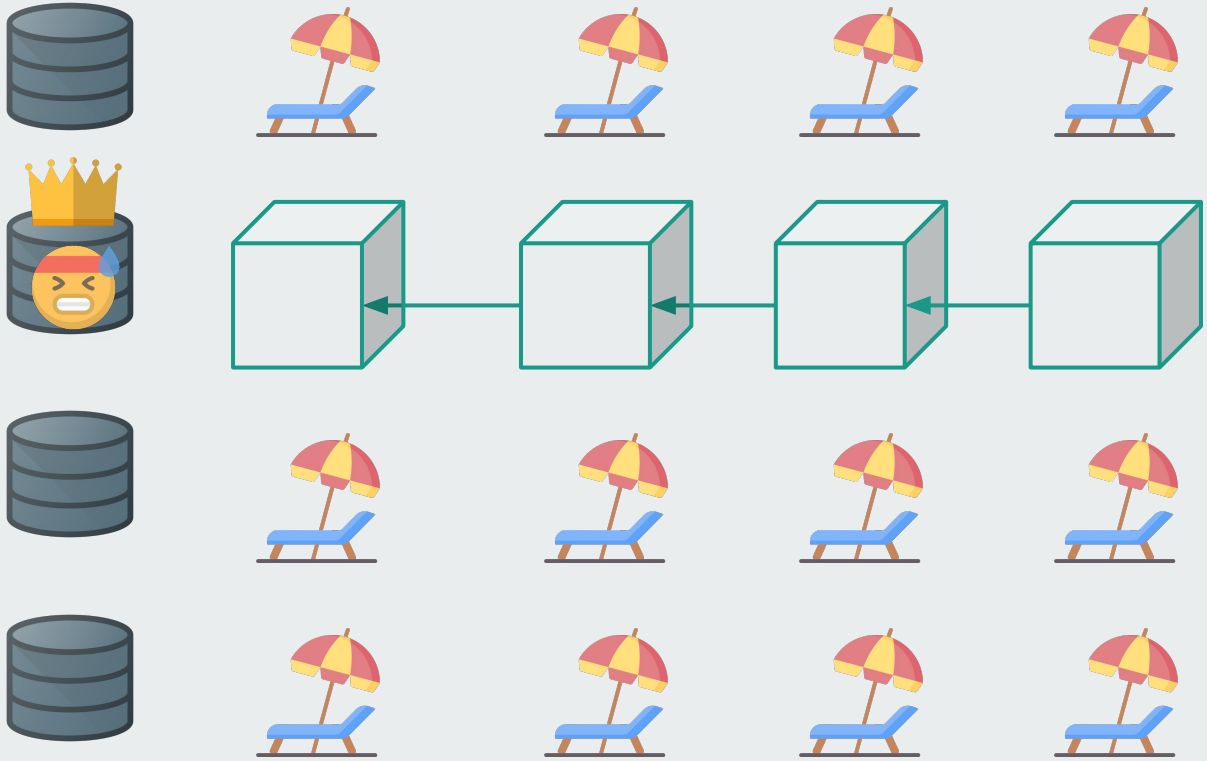


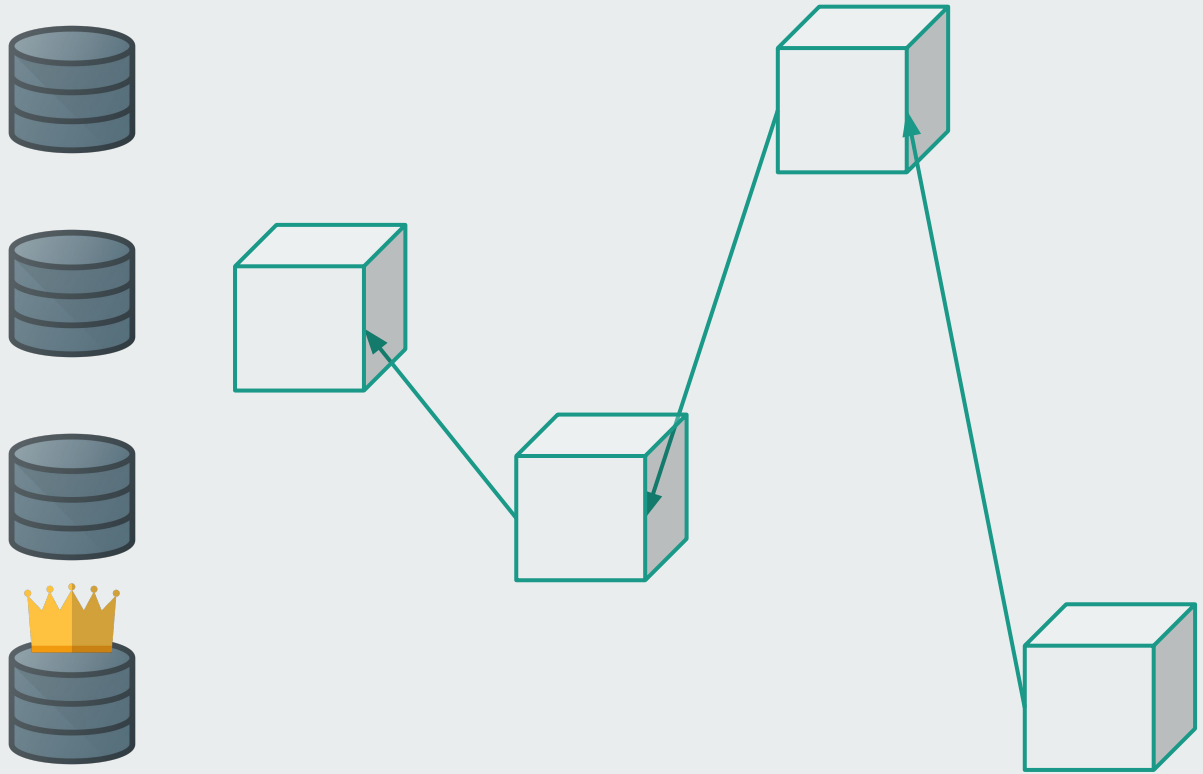
Steady Leader





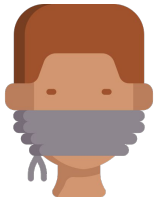






Why Rotating Leader BFT Protocols?

- Censorship resistance.



Why Rotating Leader BFT Protocols?

- Censorship resistance.
- Uniform distribution of work.



Why Rotating Leader BFT Protocols?

- Censorship resistance.
- Uniform distribution of work.
- Fairness.



Internet Computer Consensus [PODC'22]

Internet Computer Consensus

Jan Camenisch
DFINITY Foundation
Zurich, Switzerland
jan.camenisch@dfinity.org

Manu Drijvers
DFINITY Foundation
Zurich, Switzerland
manu.drijvers@dfinity.org

Timo Hanke
DFINITY Foundation
Zurich, Switzerland
timo.hanke@dfinity.org

Yvonne-Anne Pignolet
DFINITY Foundation
Zurich, Switzerland
yvonneanne.pignolet@dfinity.org

Victor Shoup
DFINITY Foundation
Zurich, Switzerland
victor.shoup@dfinity.org

Dominic Williams
DFINITY Foundation
Zurich, Switzerland
dominic.williams@dfinity.org

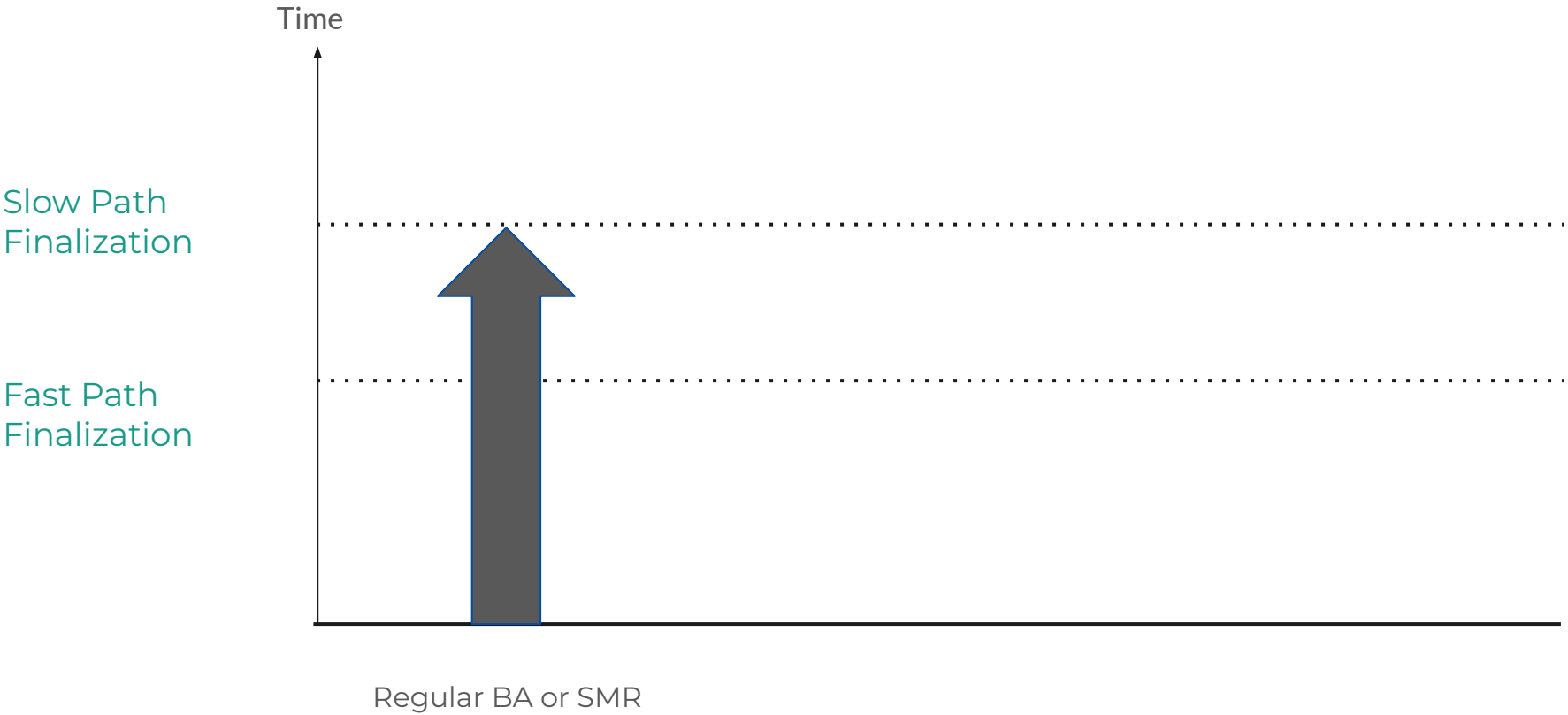
ABSTRACT

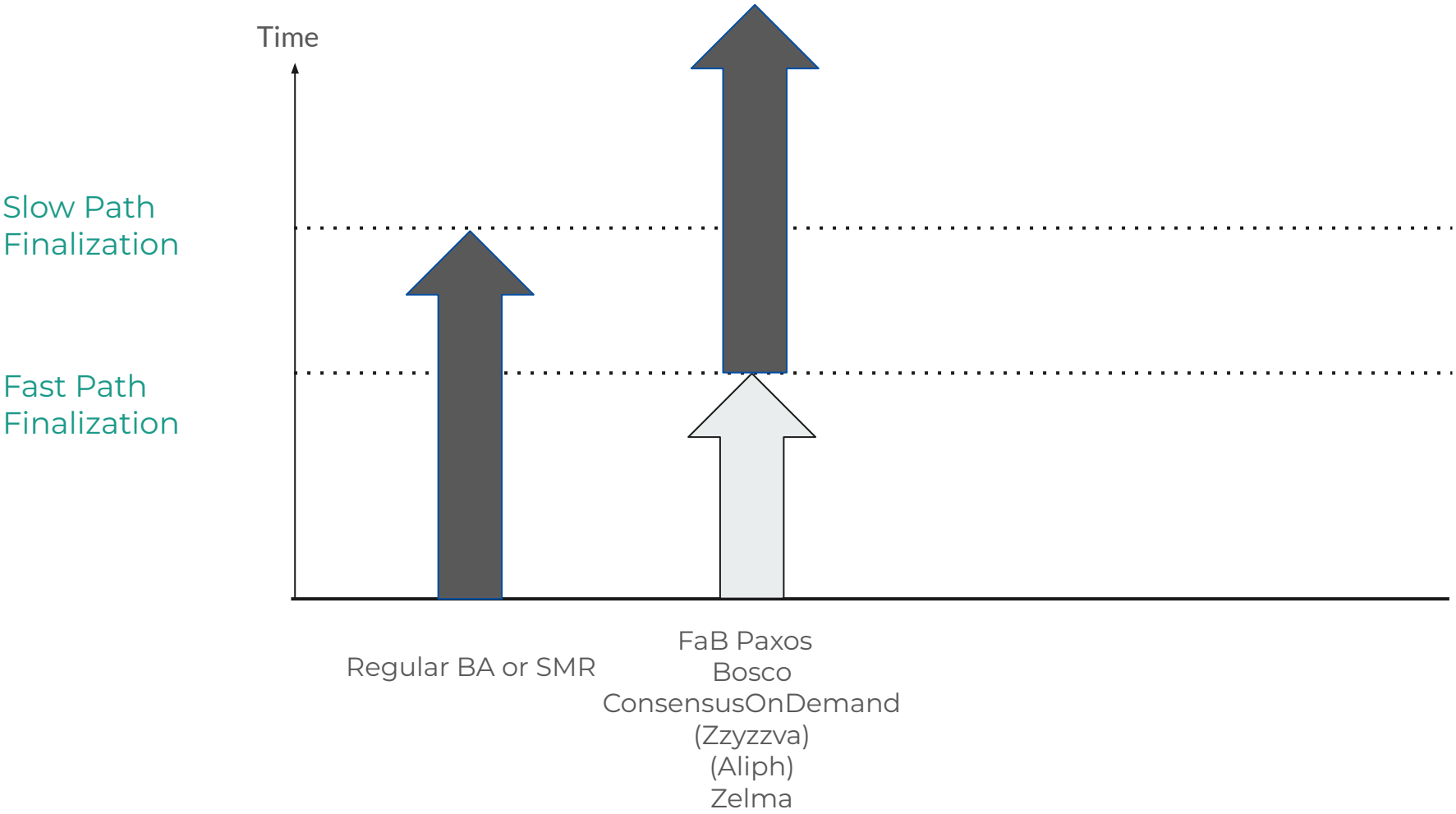
We present the Internet Computer Consensus (ICC) family of protocols for atomic broadcast (a.k.a., consensus), which underpin the Byzantine fault-tolerant replicated state machines of the Internet Computer. The ICC protocols are leader-based protocols that assume partial synchrony, and that are fully integrated with a blockchain. The leader changes probabilistically in every round. These protocols are simple and robust: in any round where the leader is corrupt (which itself happens with probability less than 1/3) or the network is asynchronous, each ICC protocol will effectively allow other parties to step in and propose blocks for that round and to move the protocol forward to the next round. In case there was no agreement on a single block in a round, a decision for this round will be taken in a later round with synchronous network behavior and an honest leader. The task of reliably disseminating the blocks to all parties is an integral part the protocol. We present three different protocols, along with various minor variations on each. The first of these protocols (ICC0) illustrates the

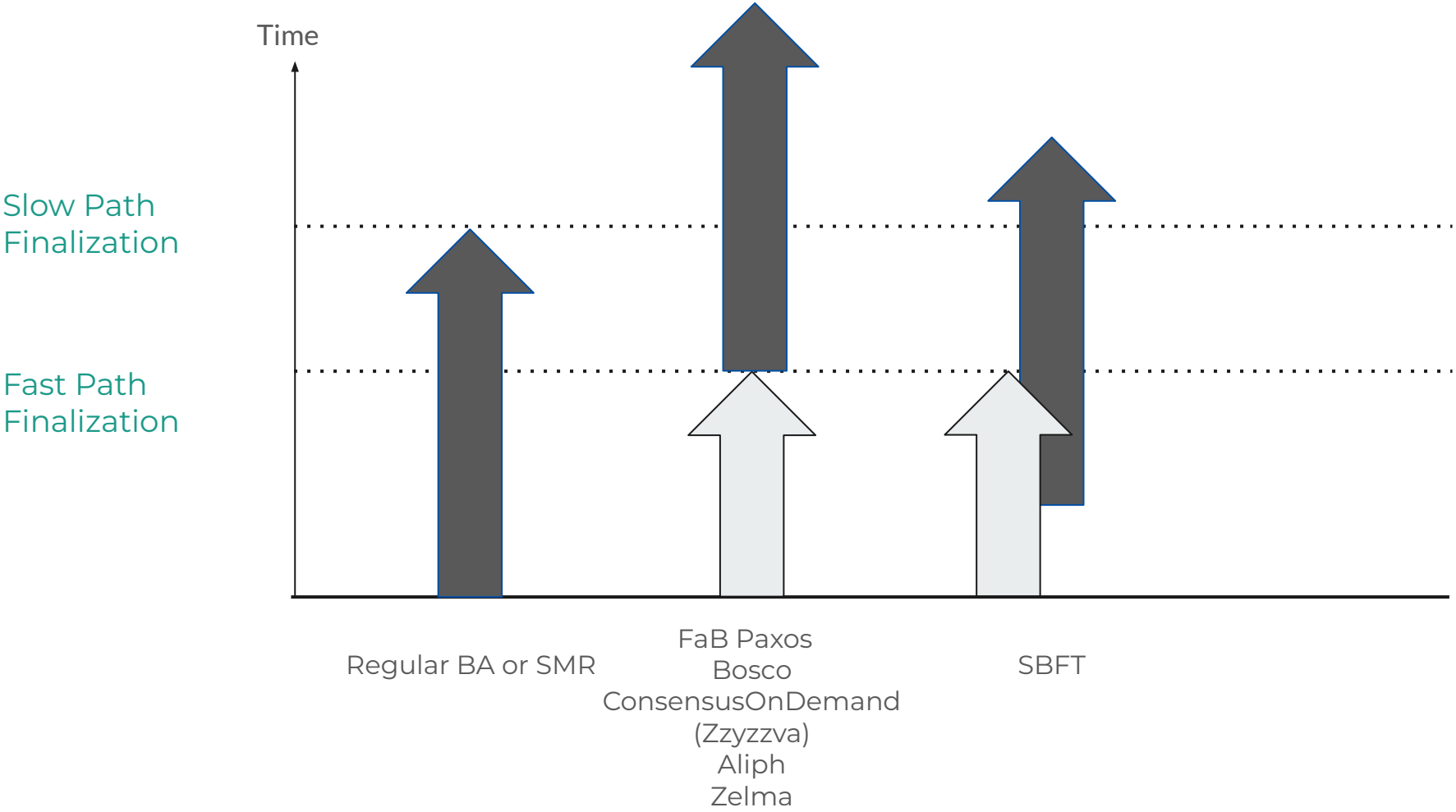
ACM Reference Format:
Jan Camenisch, Manu Drijvers, Timo Hanke, Yvonne-Anne Pignolet, Victor Shoup, and Dominic Williams. 2022. Internet Computer Consensus. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC '22), July 25–29, 2022, Salerno, Italy*. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3519270.3538430

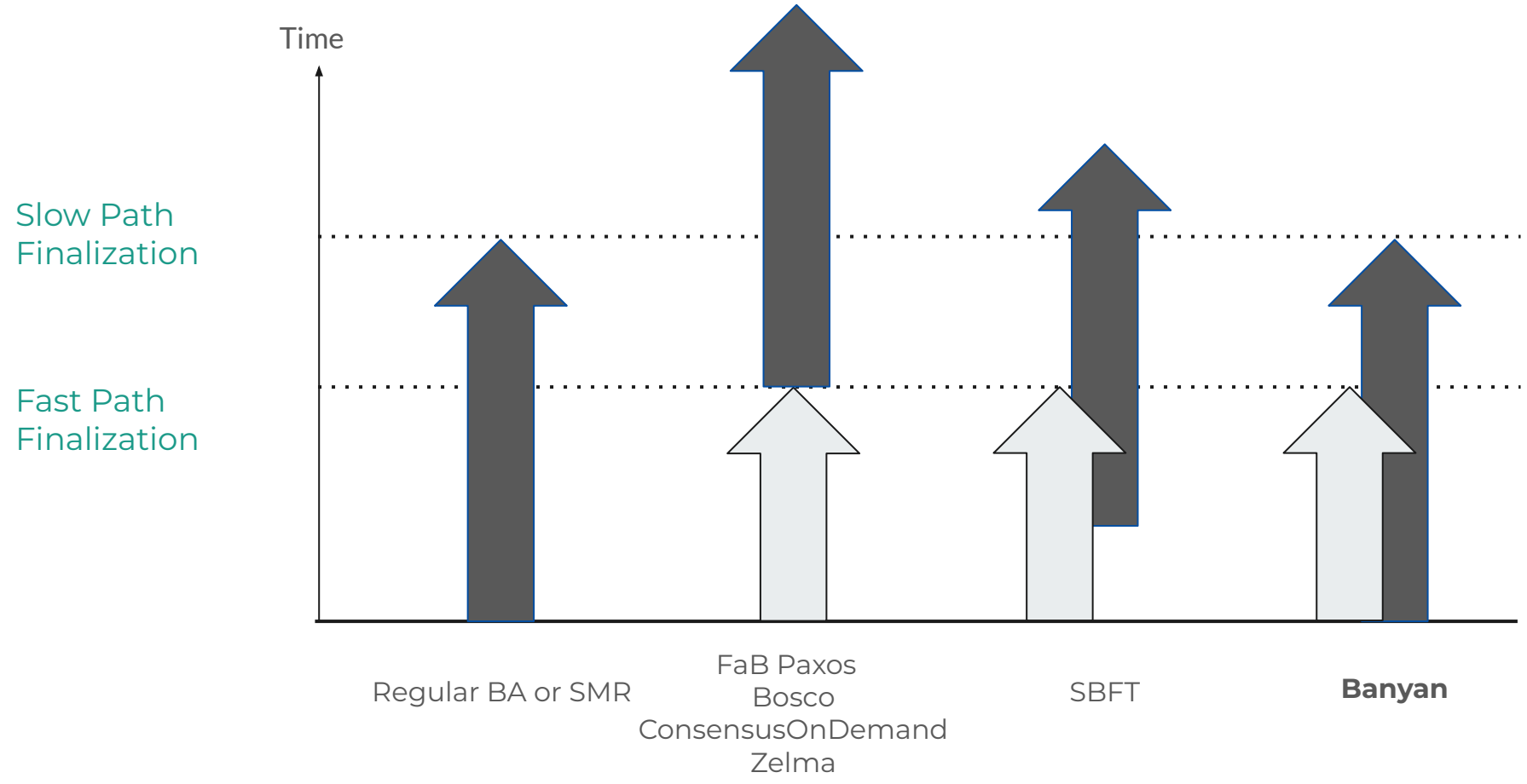
1 INTRODUCTION

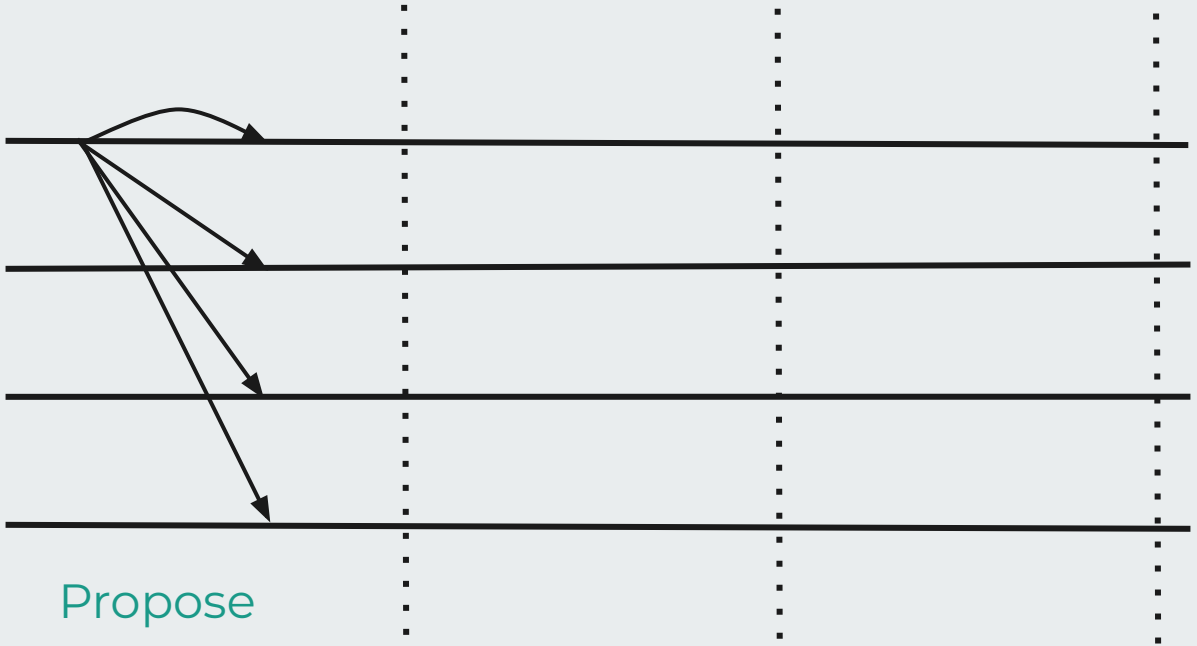
Byzantine fault tolerance (BFT) is the ability of a computing system to endure arbitrary (i.e., Byzantine) failures of some of its components while still functioning properly as a whole. One approach to achieving BFT is via *state machine replication* [33]: the logic of the system is replicated across a number of machines, each of which maintains state, and updates its state is by executing a sequence of *commands*. In order to ensure that the non-faulty machines end up in the same state, they must each deterministically execute the same sequence of commands. This is achieved by using a protocol for *atomic broadcast* [9, 16, 33].

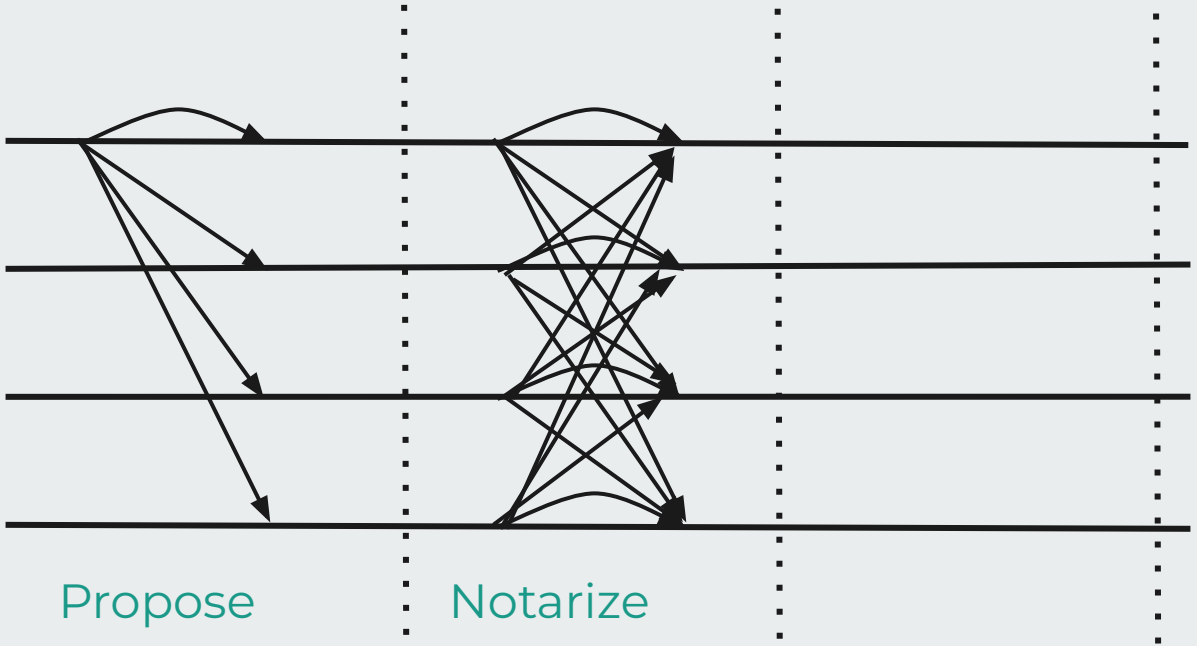

Time
Slow Path
Finalization
Fast Path
Finalization
Regular BA or SMR

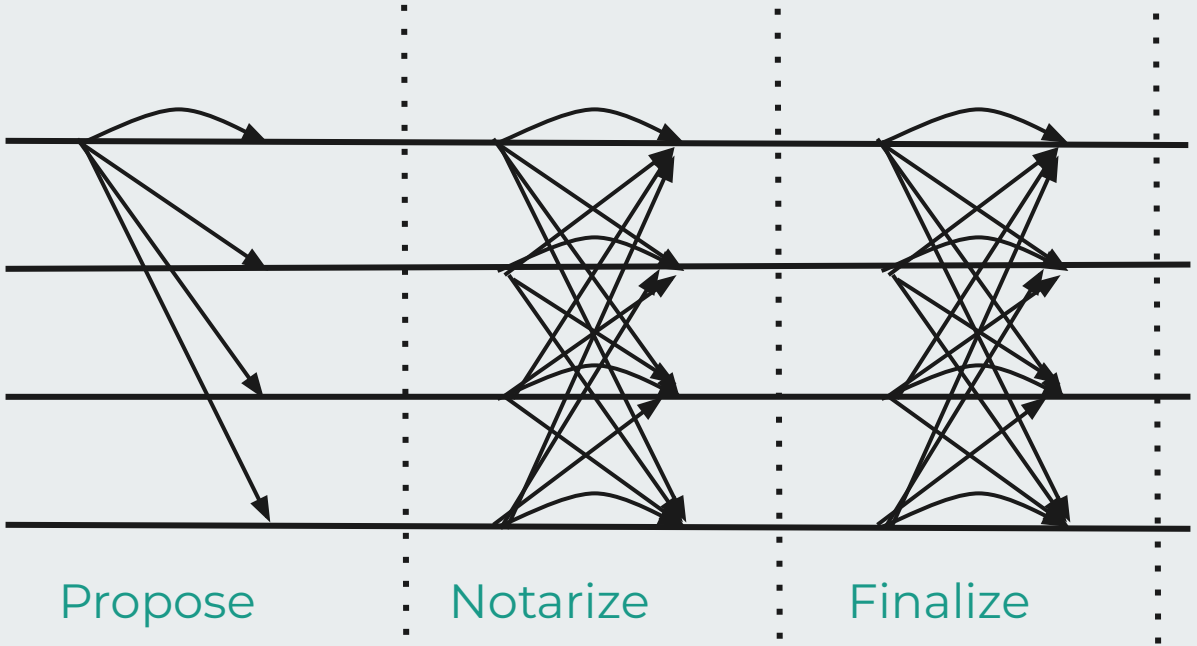

Time
Slow Path
Finalization
Fast Path
Finalization
Regular BA or SMR
FaB Paxos
Bosco
ConsensusOnDemand
(Zzyzzva)
(Aliph)
Zelma

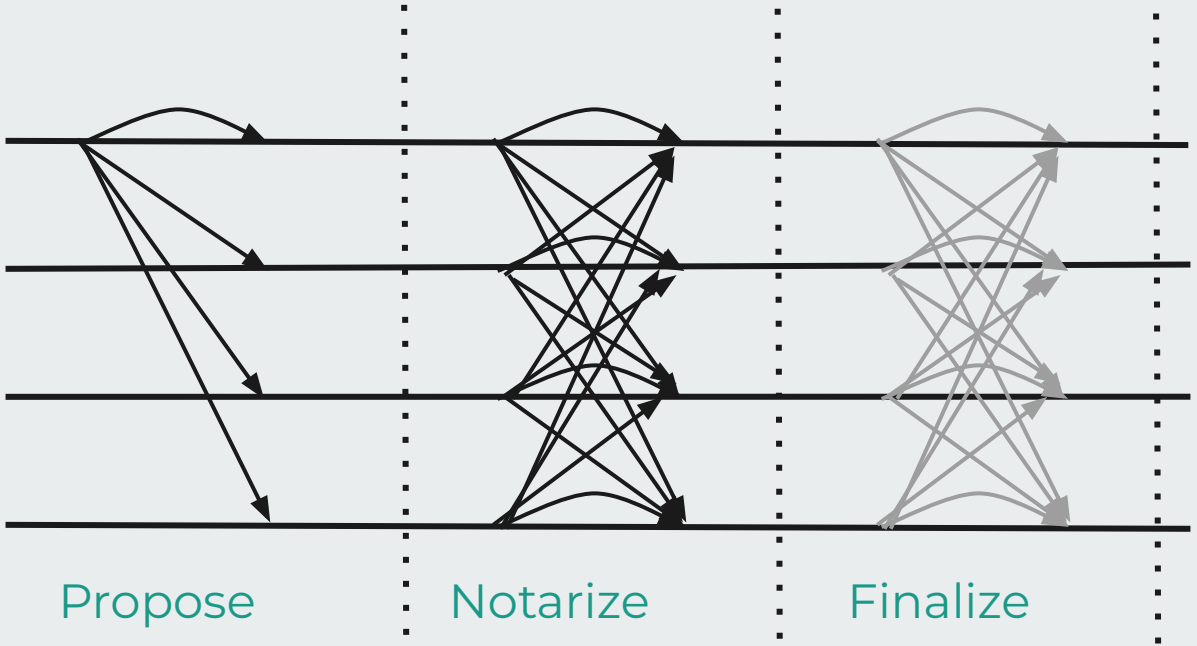

Time
Slow Path
Finalization
Fast Path
Finalization
Regular BA or SMR
FaB Paxos
Bosco
ConsensusOnDemand
(Zzyzzva)
Aliph
Zelma
SBFT


Time
Slow Path Finalization
Fast Path Finalization
Regular BA or SMR
FaB Paxos
Bosco
ConsensusOnDemand
Zelma
SBFT
Banyan


Propose


Propose
Notarize


Propose
Notarize
Finalize

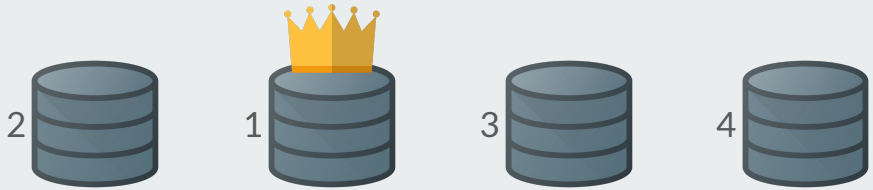

Propose
Notarize
Finalize

Internet Computer Consensus - Optimistic Case

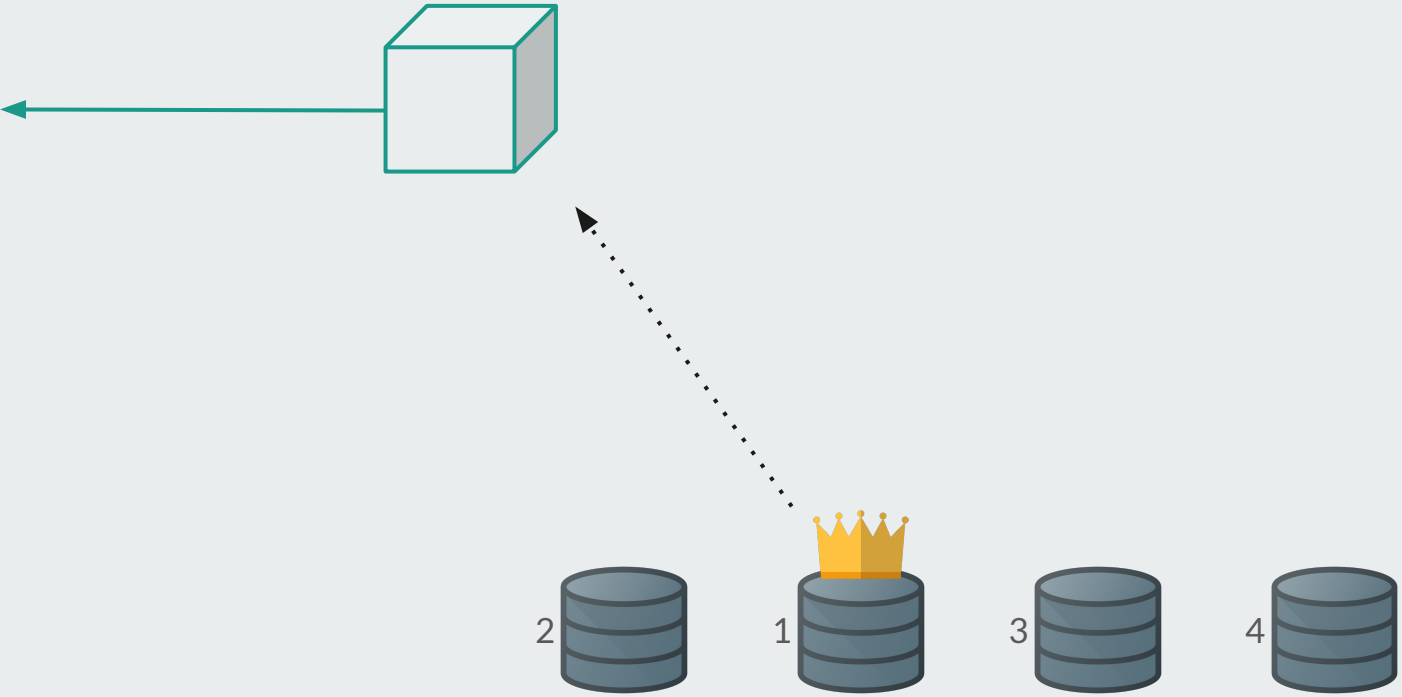


Internet Computer Consensus - Optimistic Case

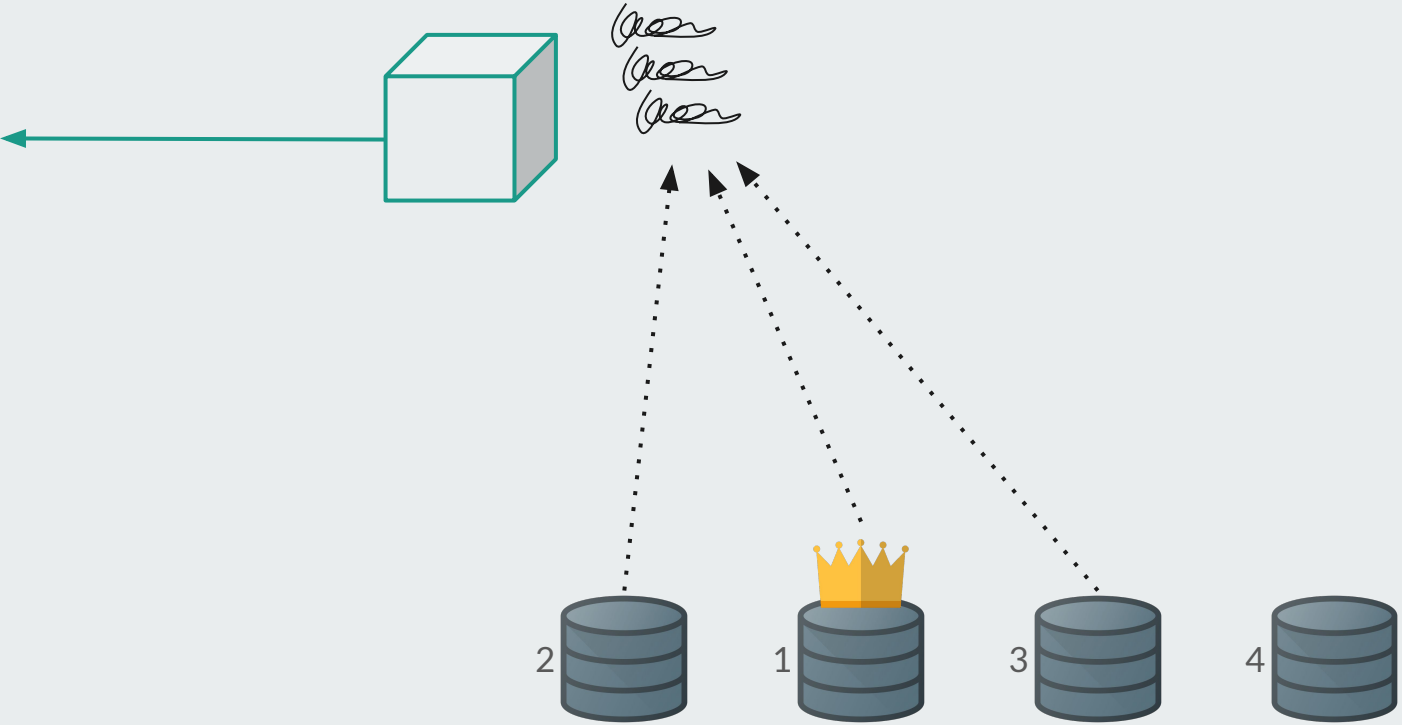




Internet Computer Consensus - Optimistic Case

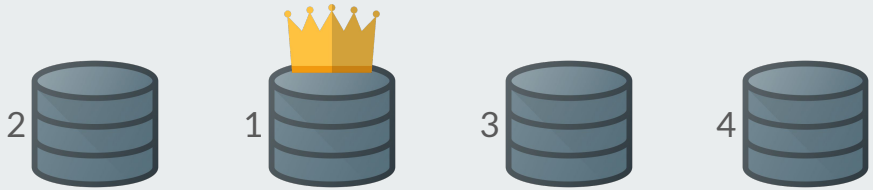


Internet Computer Consensus - Optimistic Case

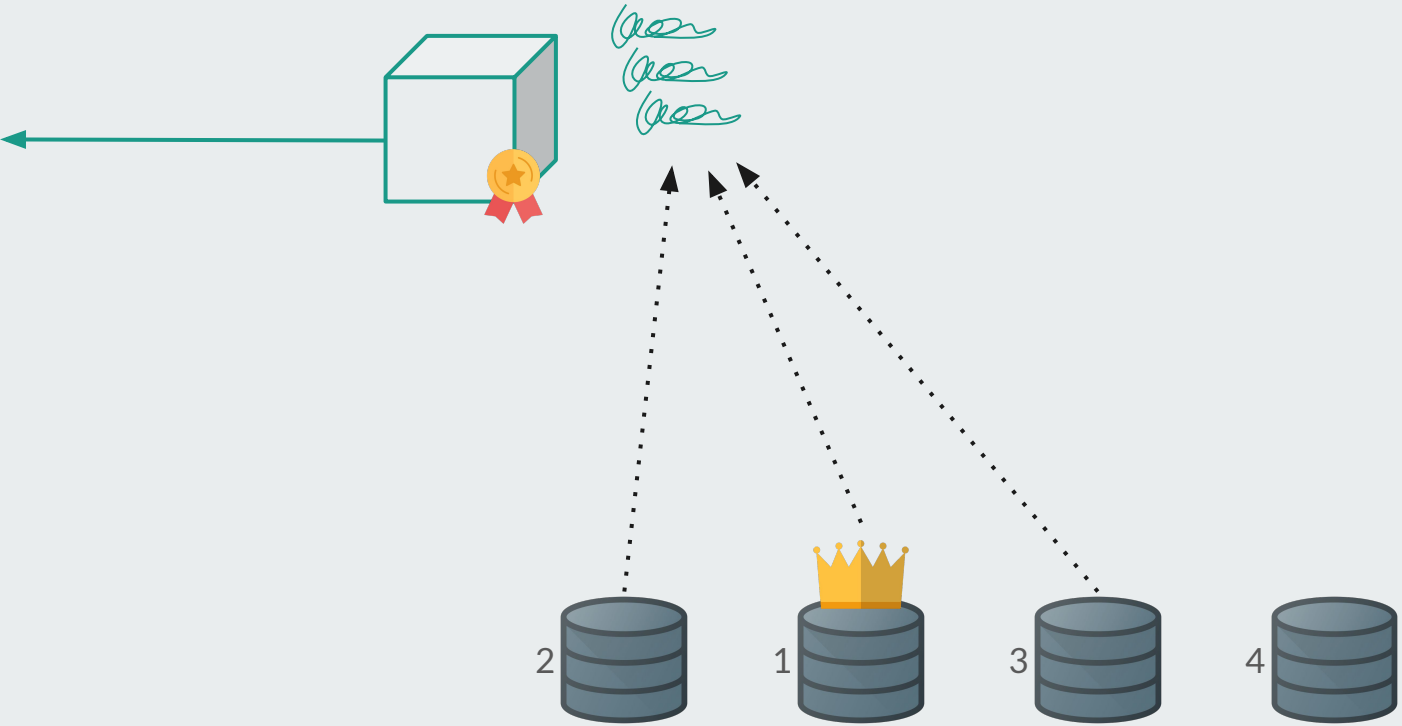


Internet Computer Consensus - Optimistic Case



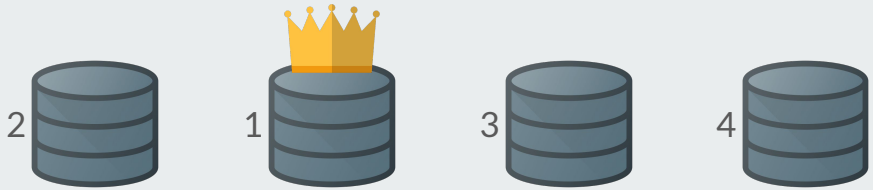


Internet Computer Consensus - Optimistic Case



Internet Computer Consensus - Optimistic Case

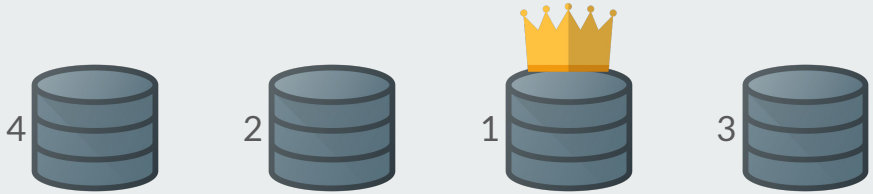




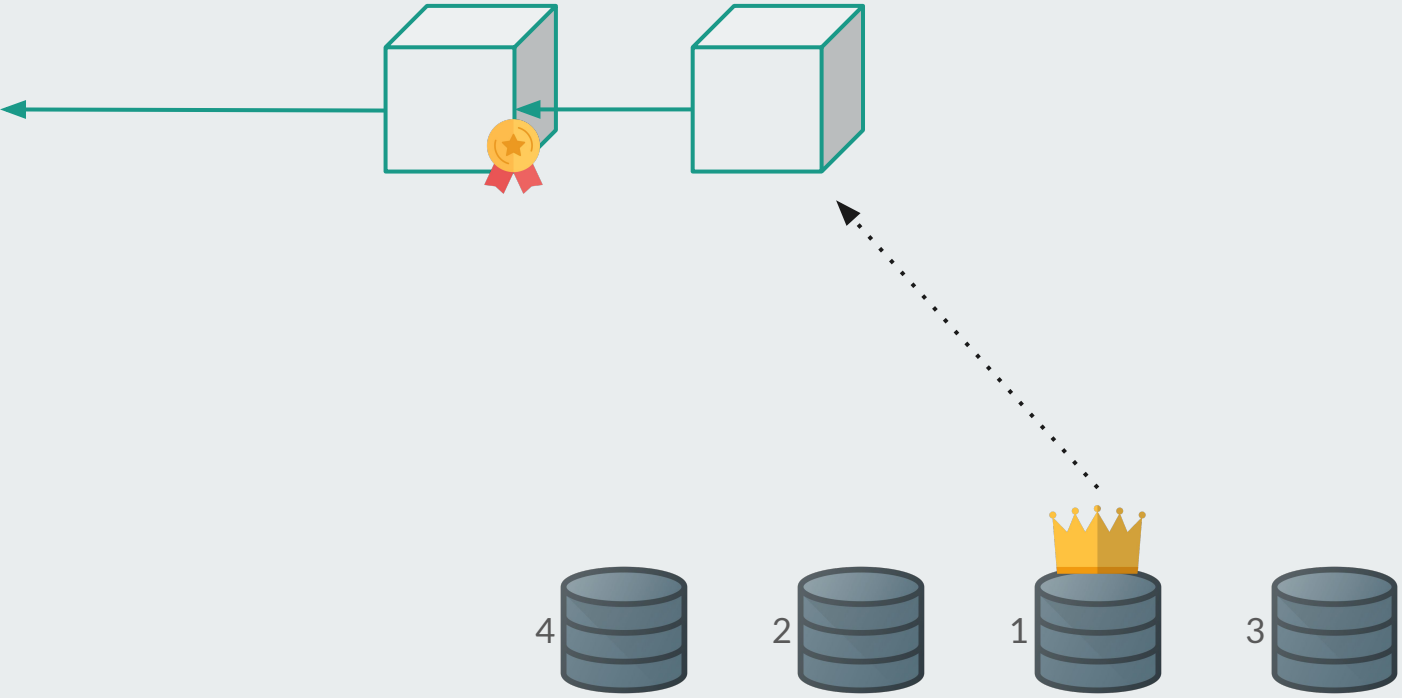
Internet Computer Consensus - Optimistic Case



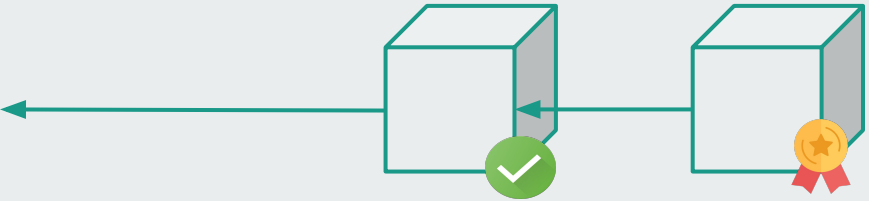


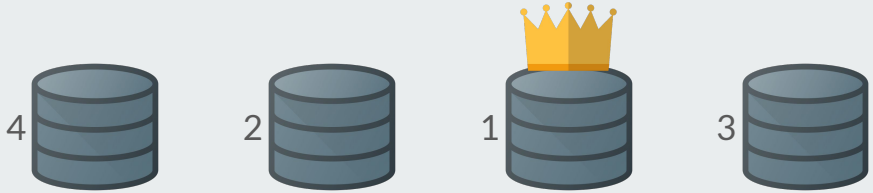


Internet Computer Consensus

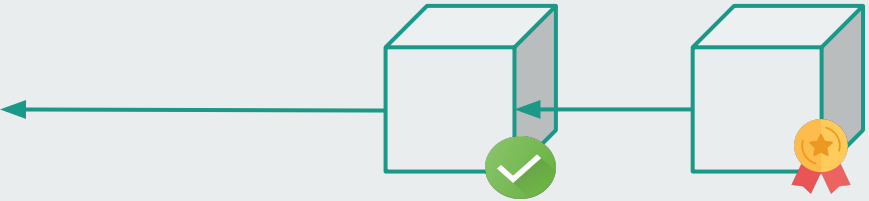


Internet Computer Consensus





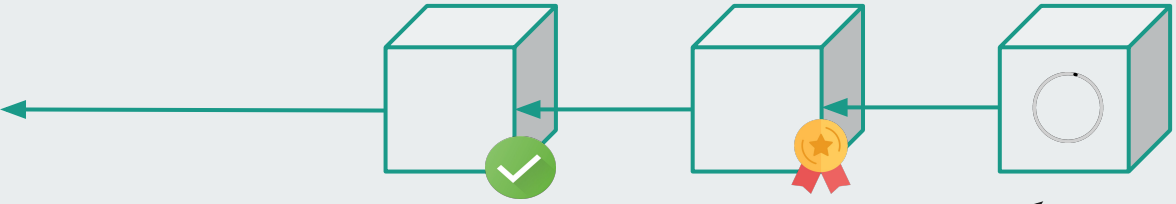
Internet Computer Consensus







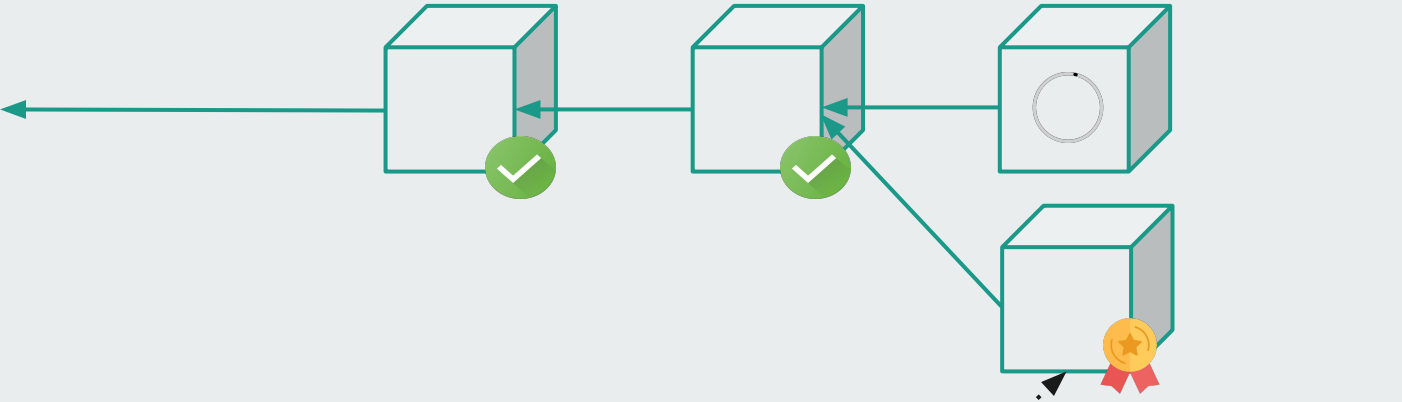
Internet Computer Consensus

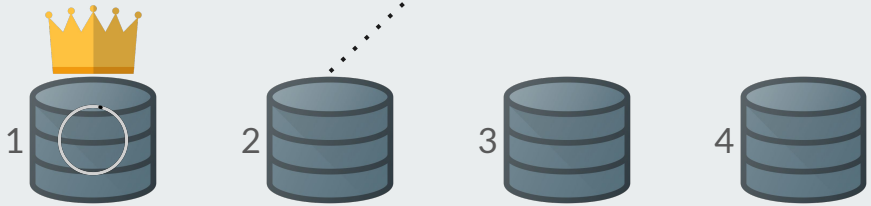




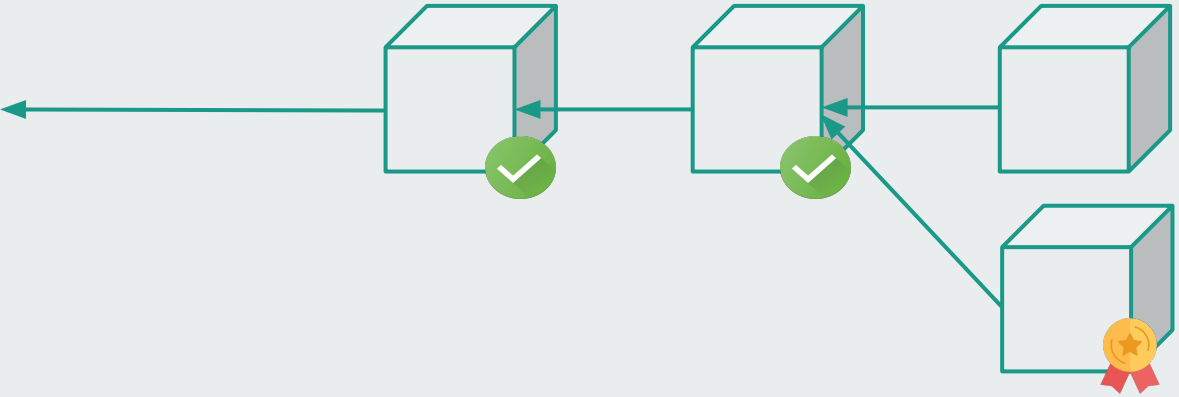


Internet Computer Consensus



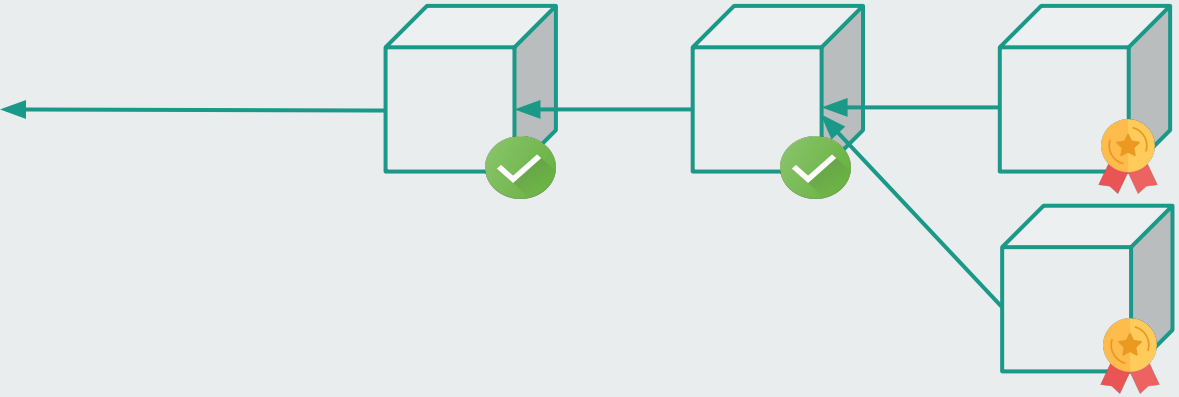


Internet Computer Consensus





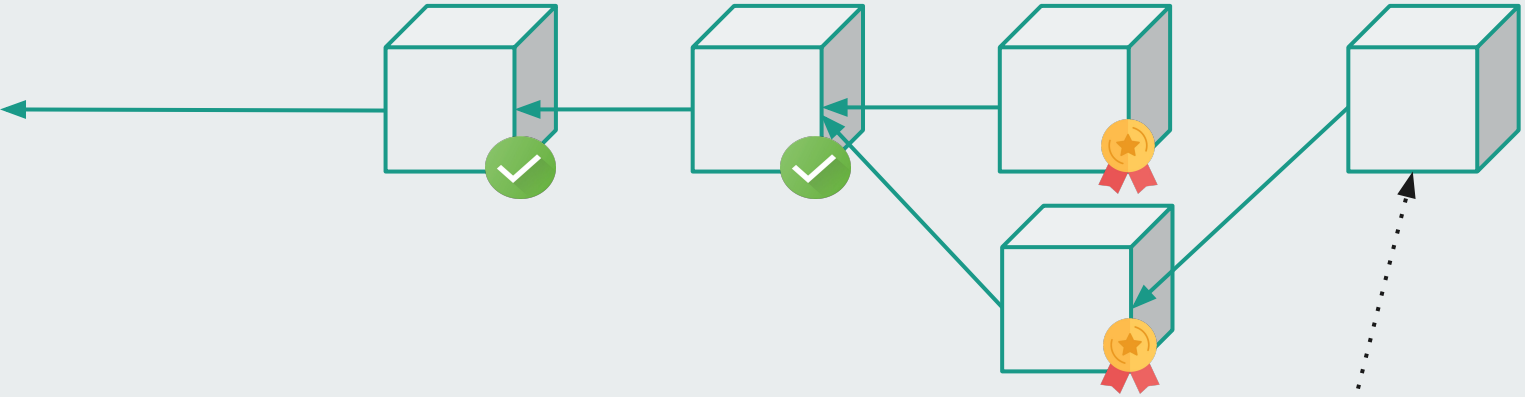
Internet Computer Consensus







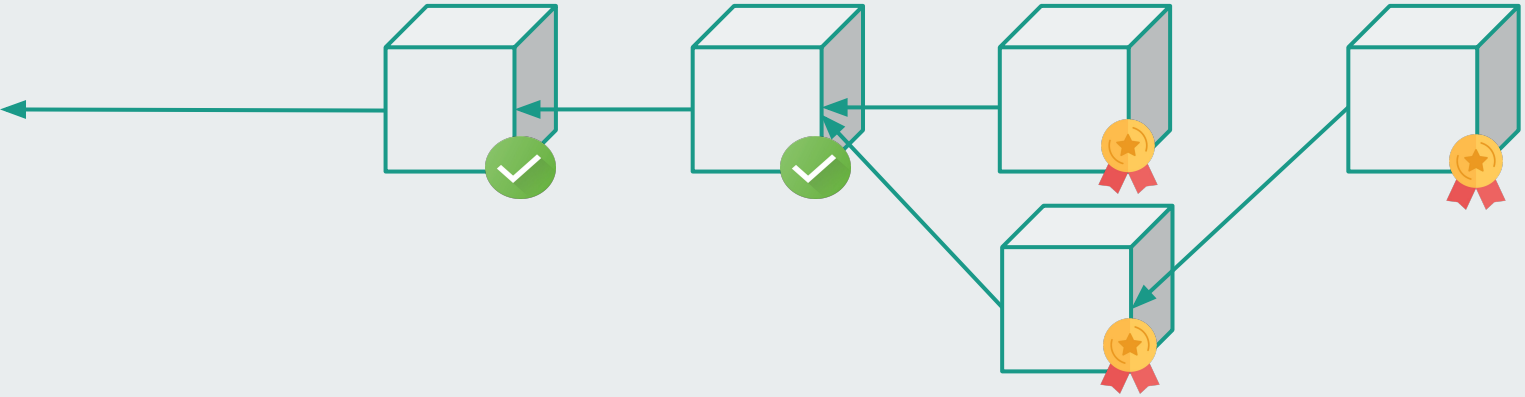
Internet Computer Consensus





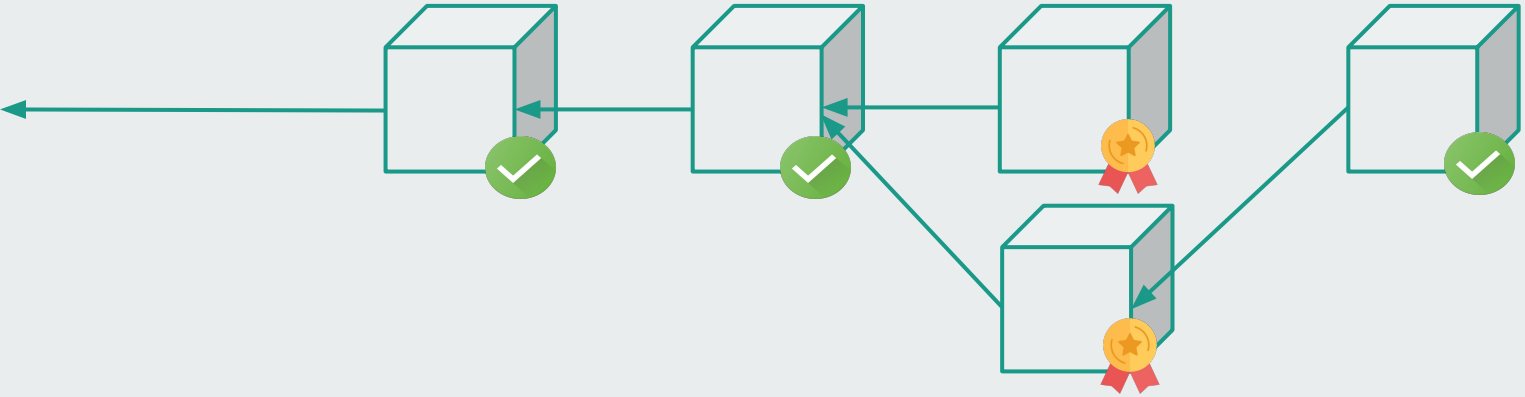


Internet Computer Consensus



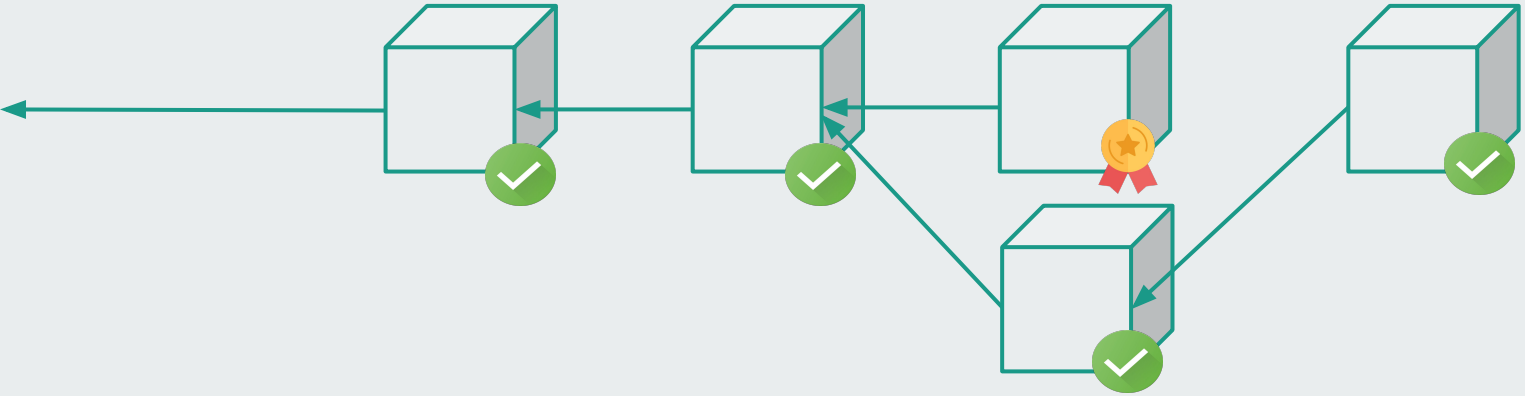


Internet Computer Consensus





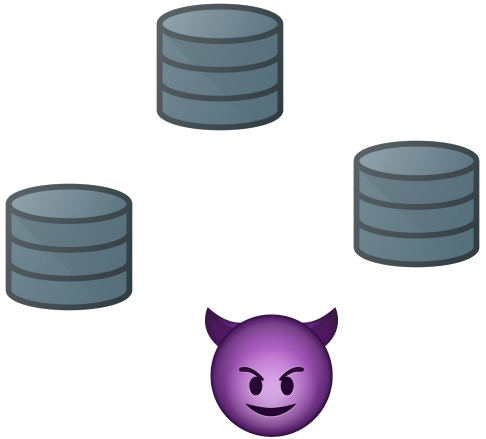
Internet Computer Consensus





Model – ICC

$n \geq 3f + 1$



Internet Computer Consensus

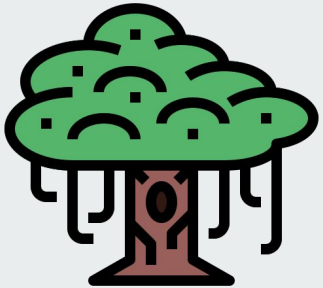


notarization signatures



finalization signatures

Banyan



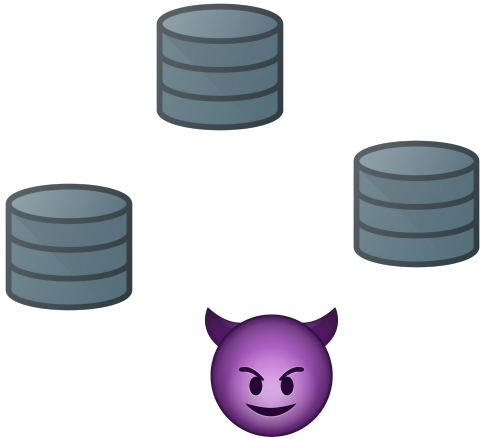
first notarization signature

other notarization signatures

finalization signatures

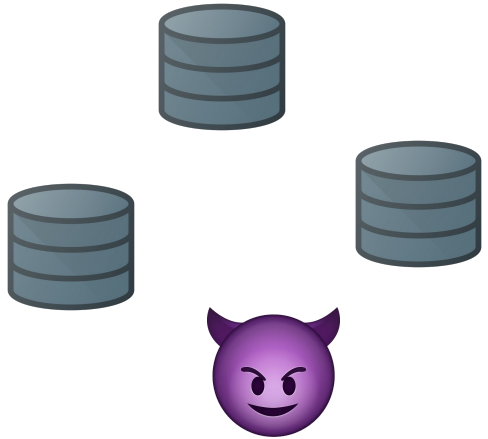
Model – Banyan

$n \geq 3f + 2p^* \mathbf{- 1}$



Model – Banyan

$n \geq 3f + 2p^* - 1$



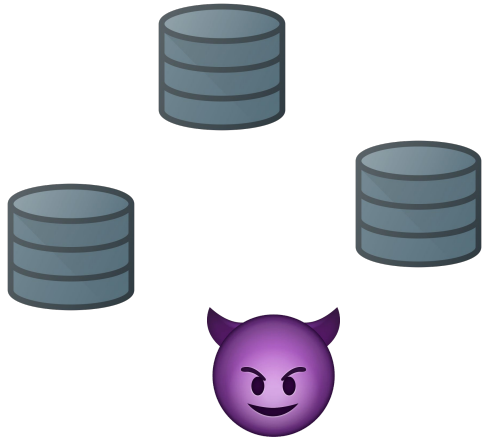
Rule: Commit when **(n - p)**



notarization signatures received

Model – Banyan

We get p=1 for free!!

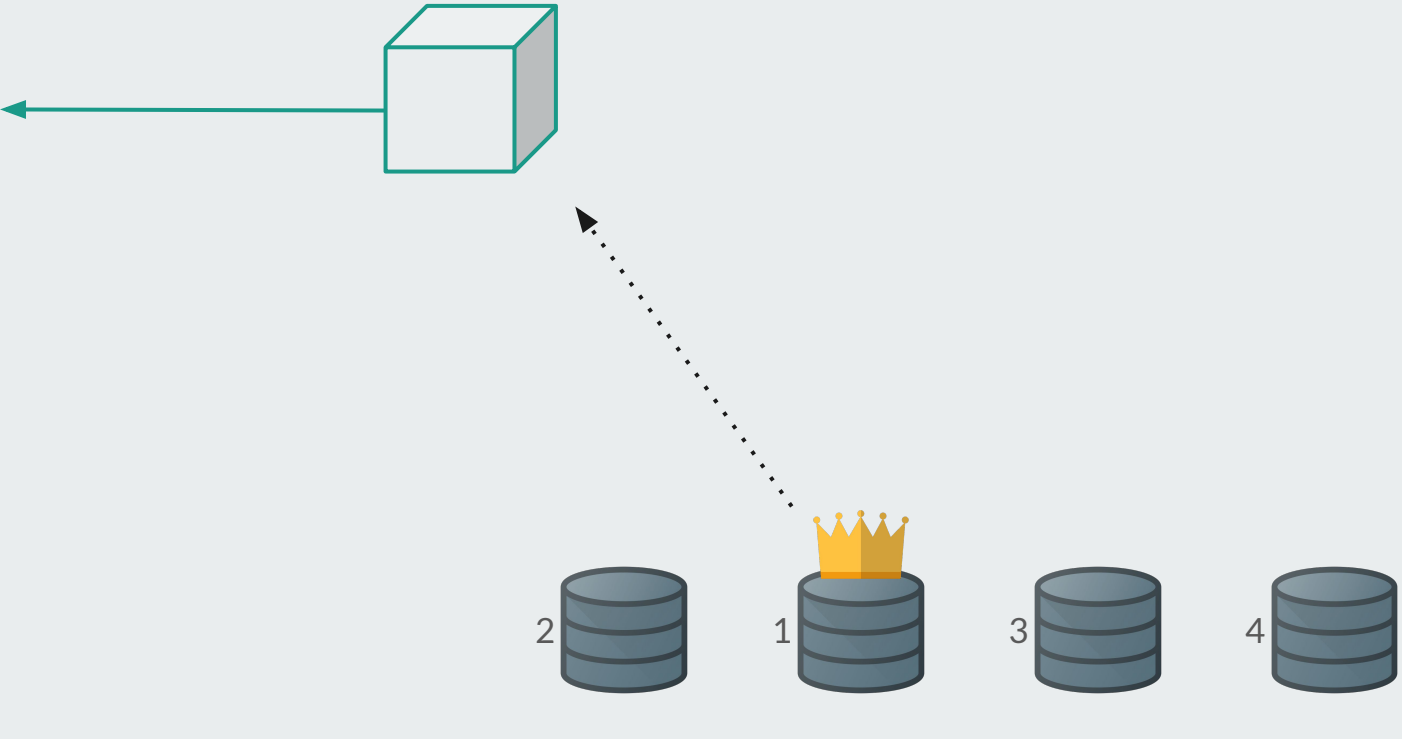


$$n \geq 3f + 2p^* \textbf{ - 1}$$

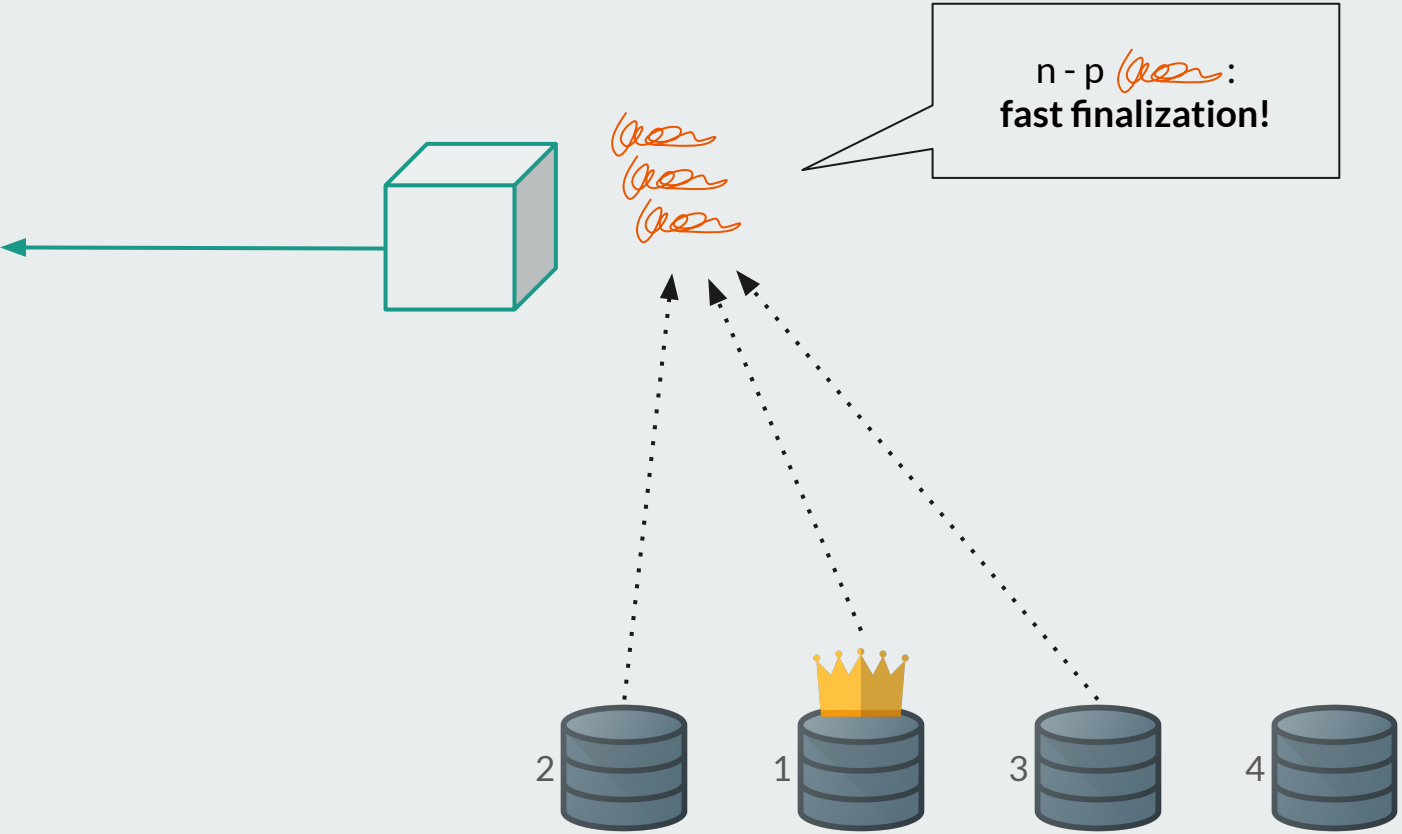
Rule: Commit when **(n - p)**

notarization signatures received

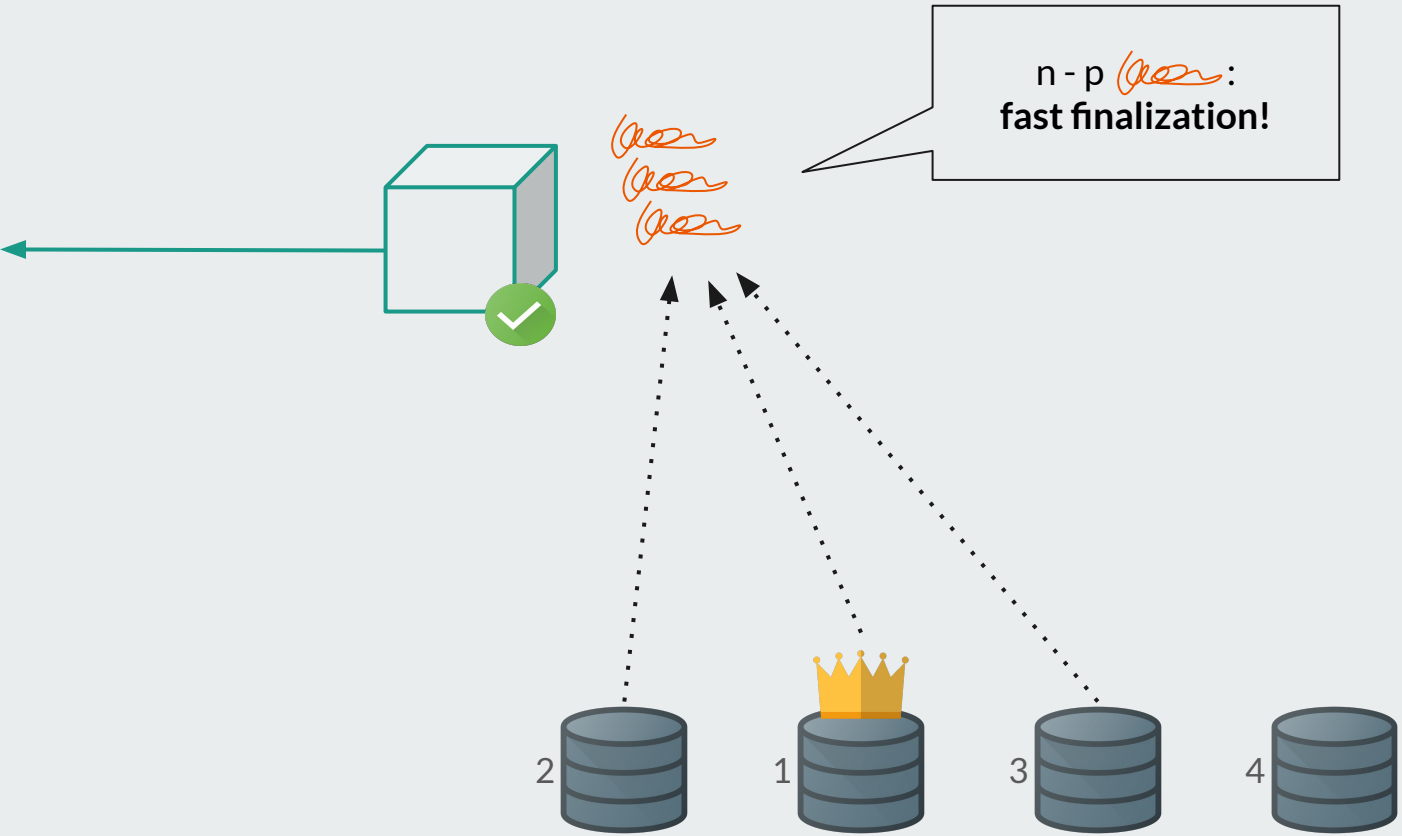
Banyan - Optimistic Case



Banyan - Optimistic Case

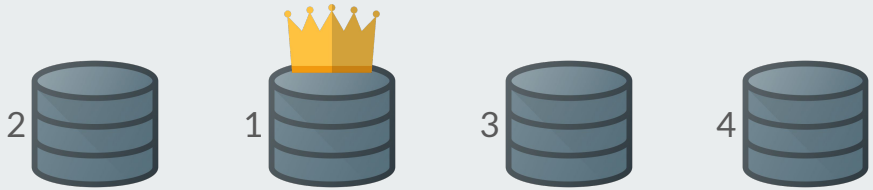


Banyan - Optimistic Case



Banyan - Optimistic Case

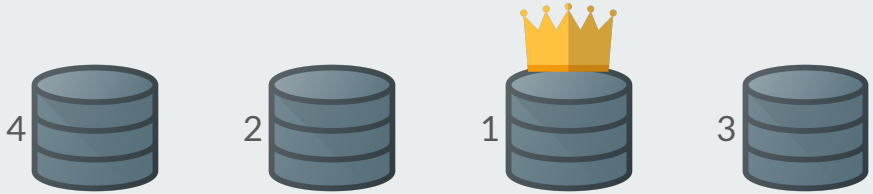




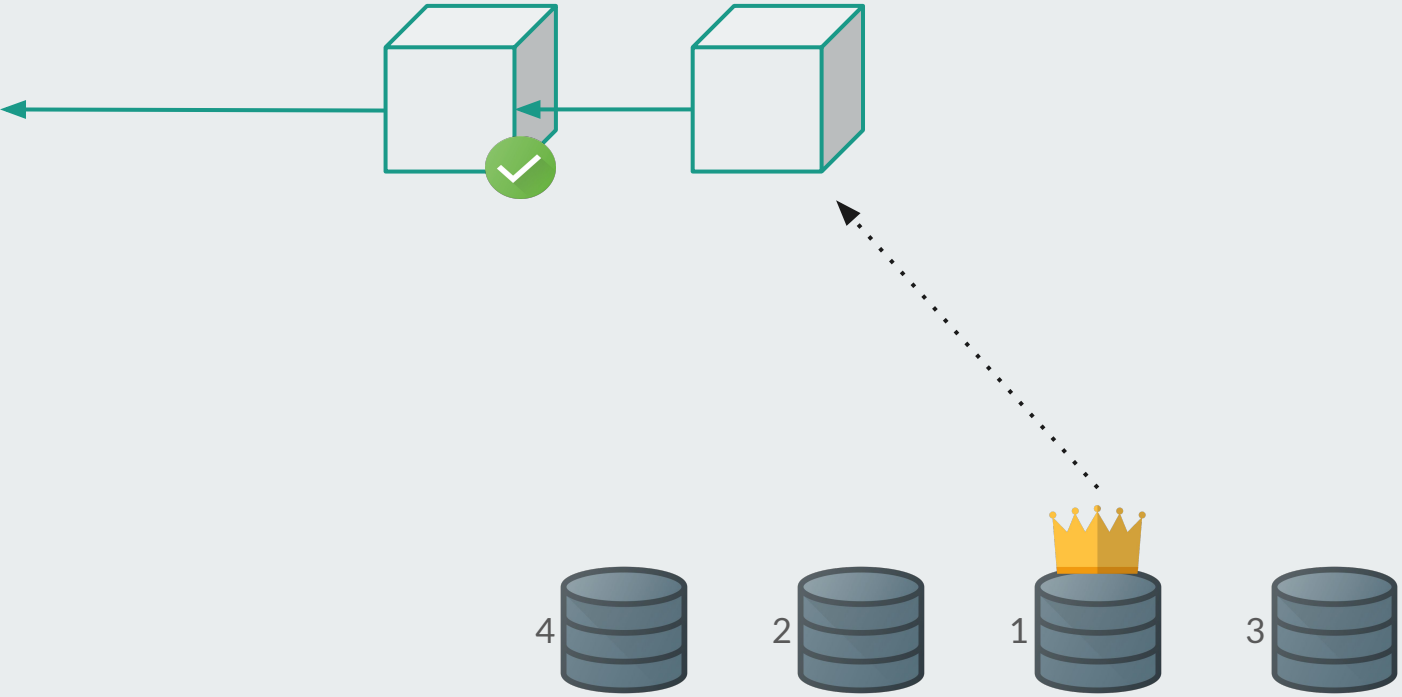
Banyan - Optimistic Case



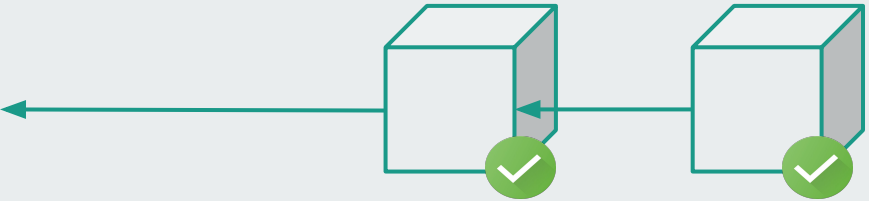


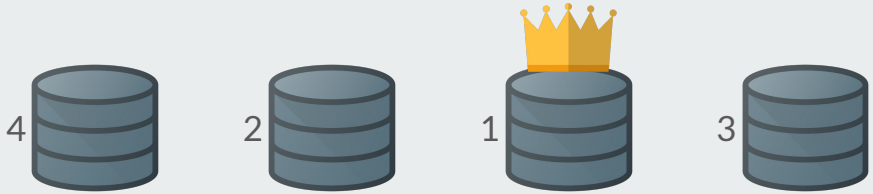


Banyan - Optimistic Case

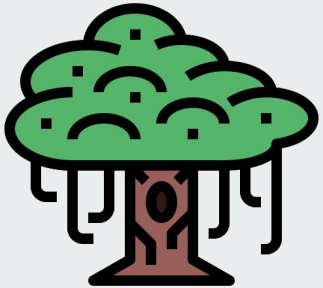


Banyan - Optimistic Case





Banyan - Recap



Fast path: Commit when **n - p**



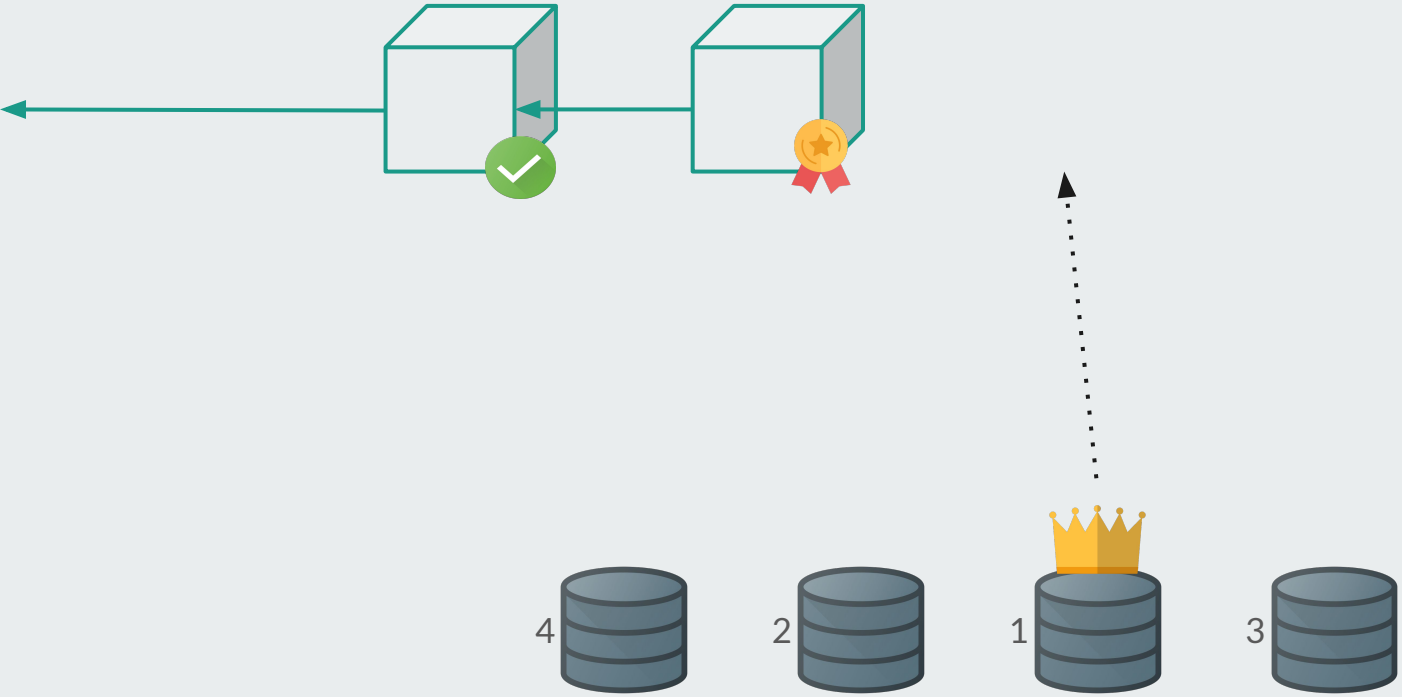
notarization signatures received

Slow path: ICC + new rule:

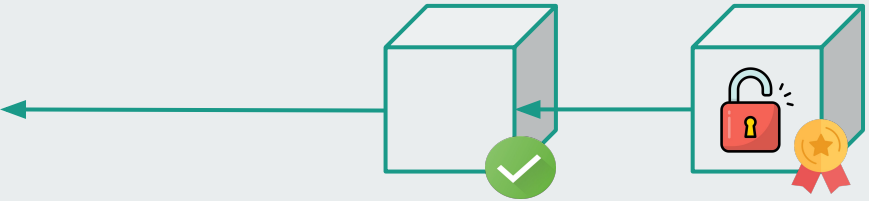


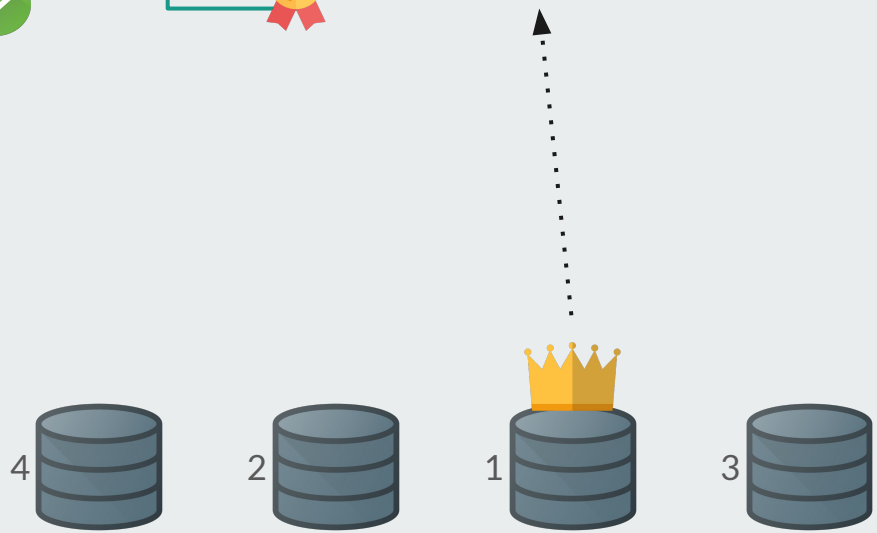
only extend and finalize *unlocked* blocks

Banyan - Unlocking Blocks

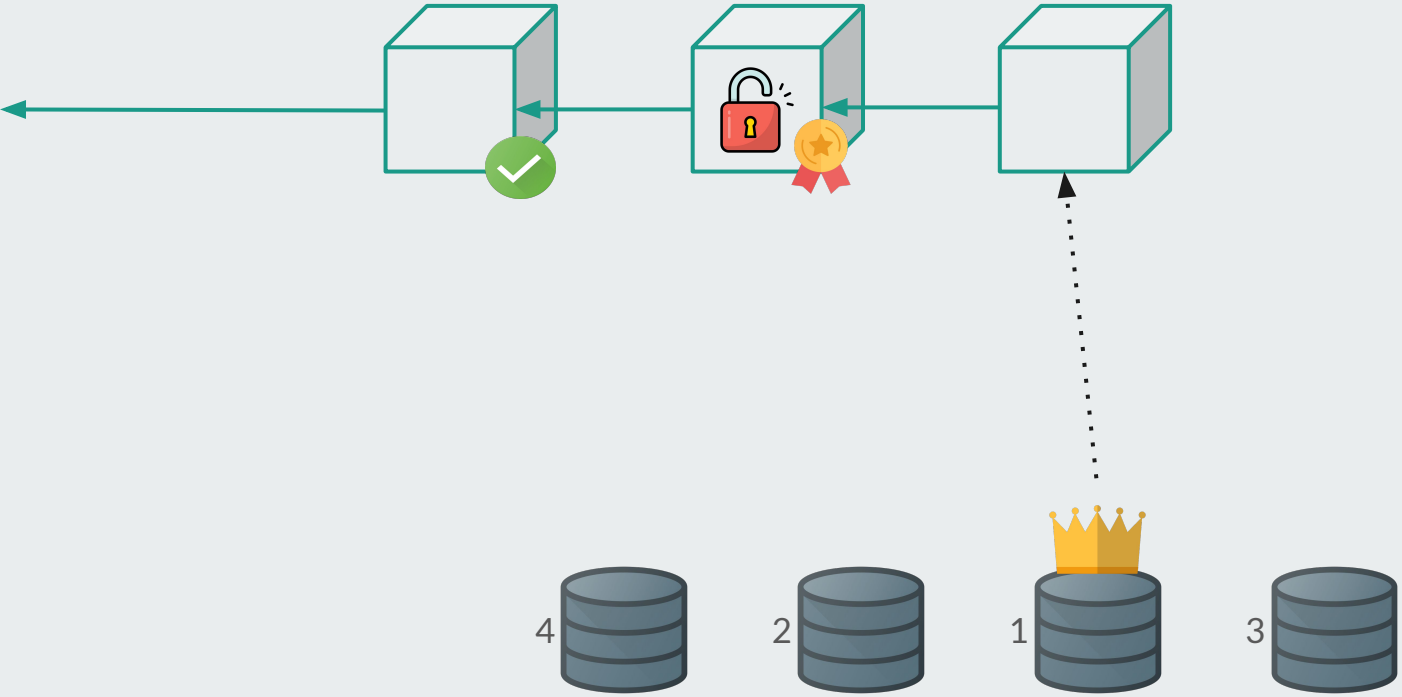


Banyan - Unlocking Blocks

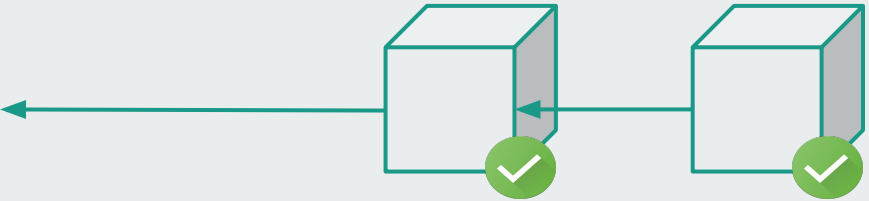


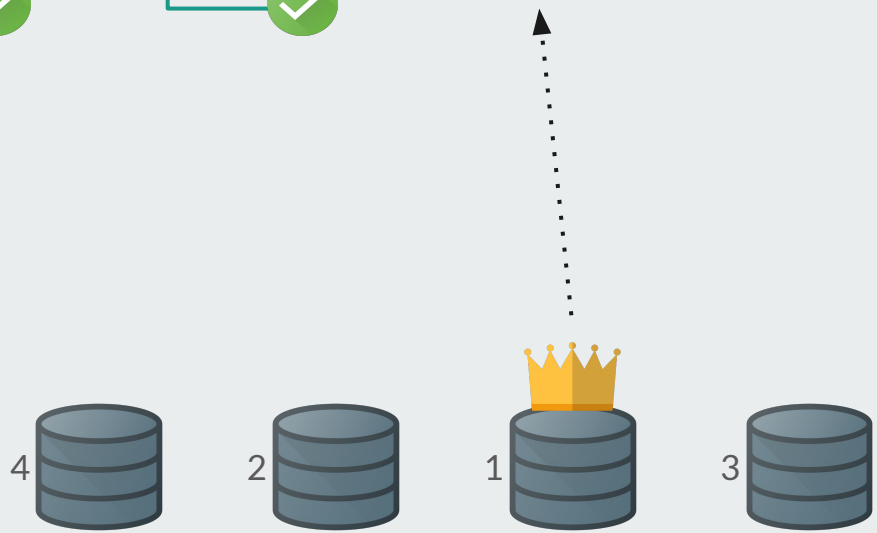


Banyan - Unlocking Blocks

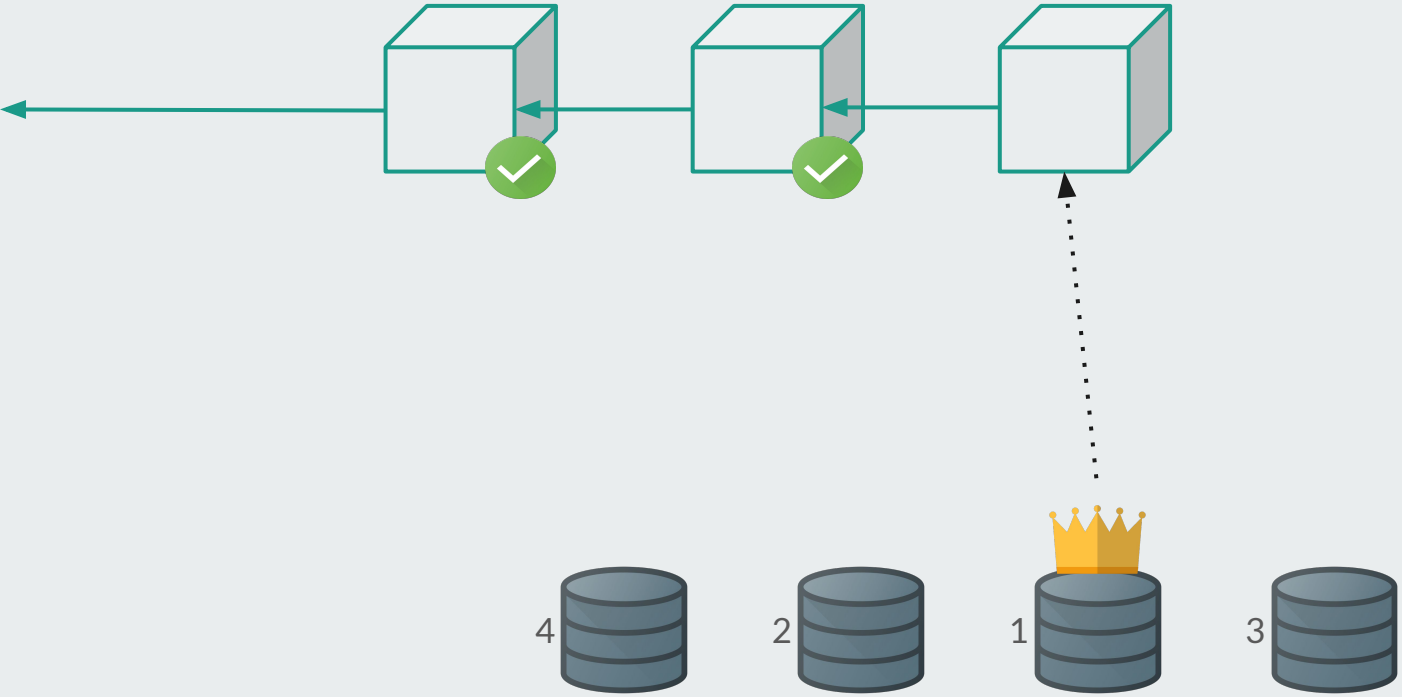


Banyan - Unlocking Blocks



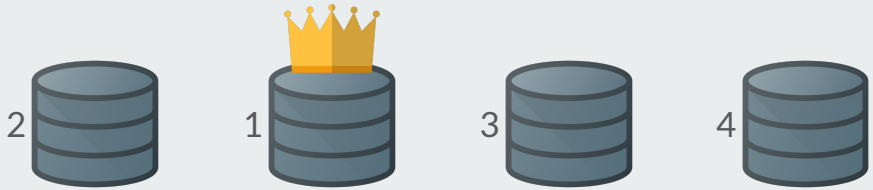


Banyan - Unlocking Blocks

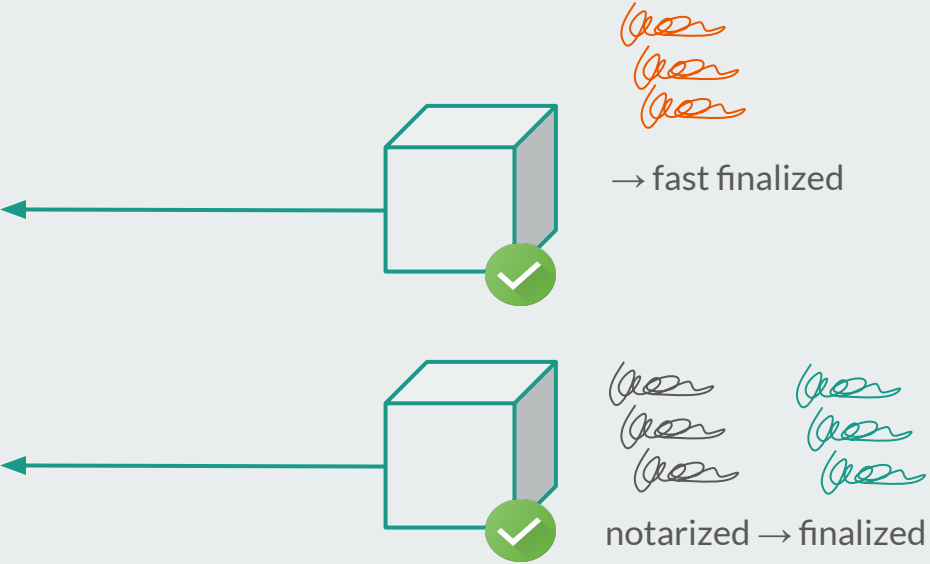


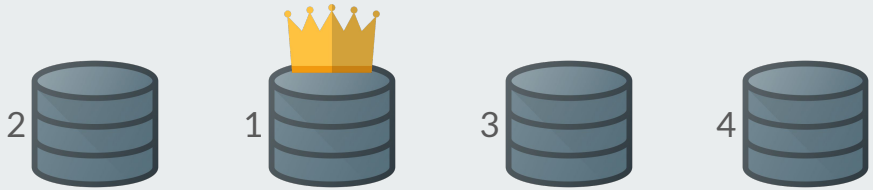
Unlocked Block: Will not conflict with fast path



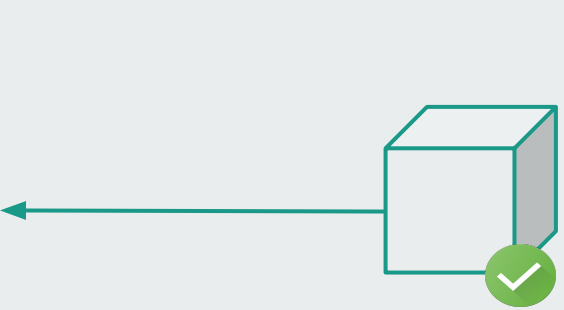


Correctness

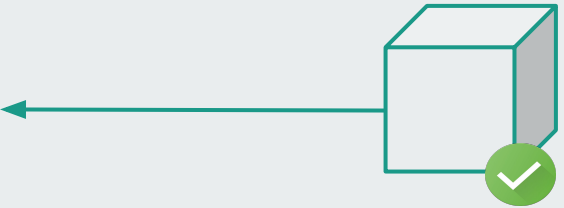




Correctness

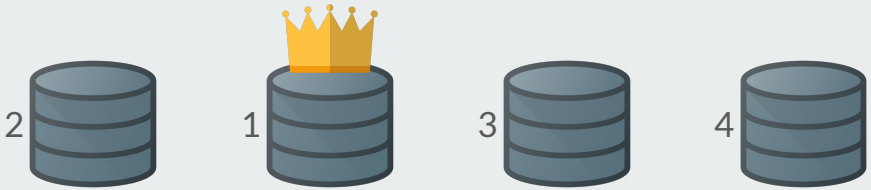


→ fast finalized

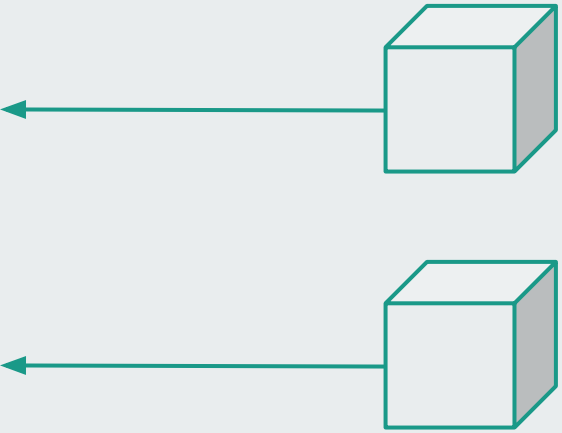


notarized → finalized

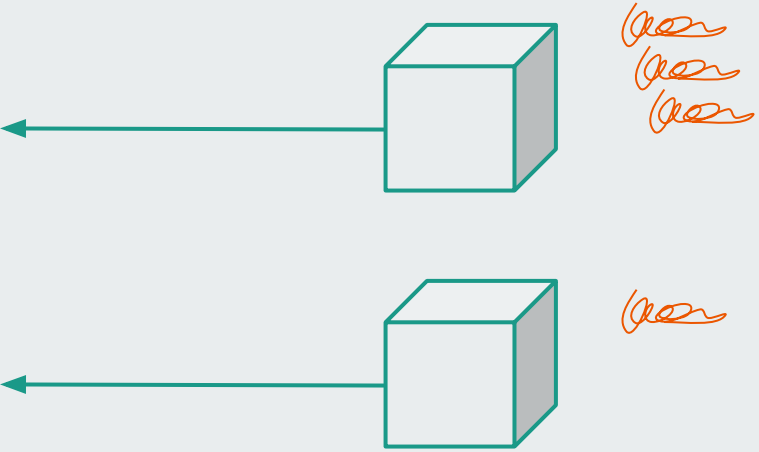




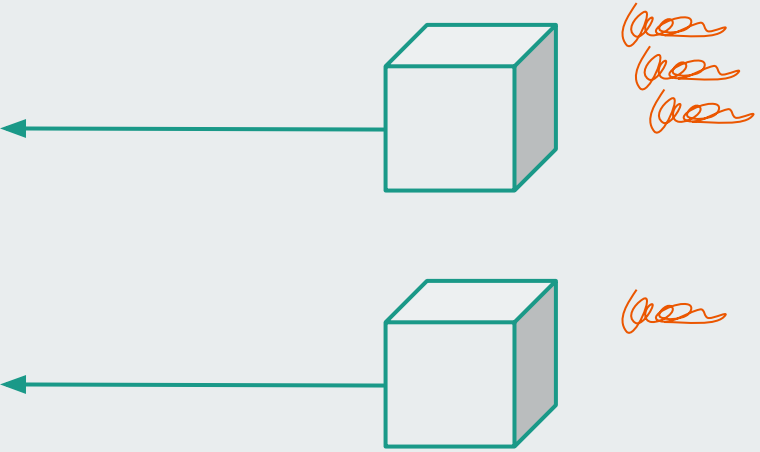
Unlocked Block - Case 1



Unlocked Block - Case 1



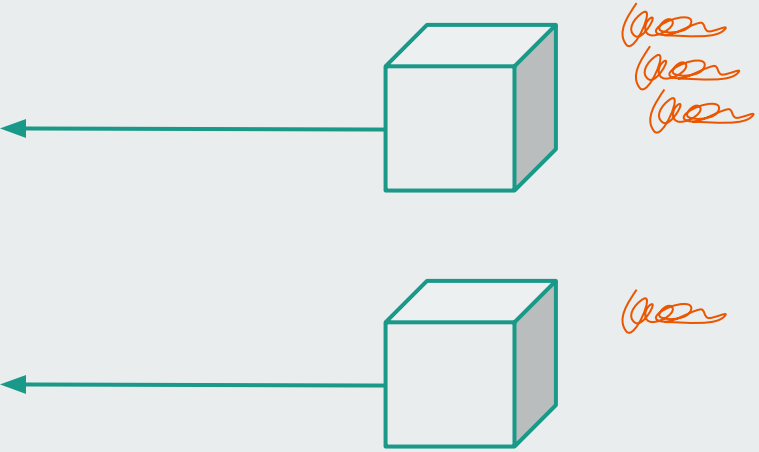
Unlocked Block - Case 1



number of *for block B* **> *f* + *p***

→ B unlocked

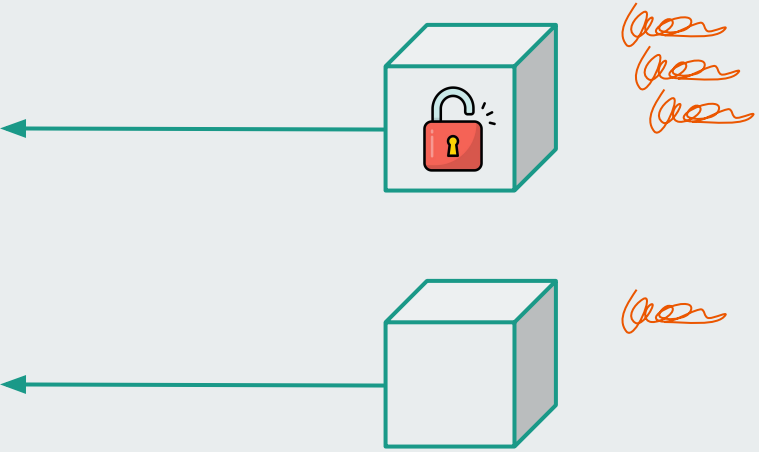
Unlocked Block - Case 1



number of *for block B*
+ number of *for non leader blocks* **> f + p**

→ B unlocked

Unlocked Block - Case 1

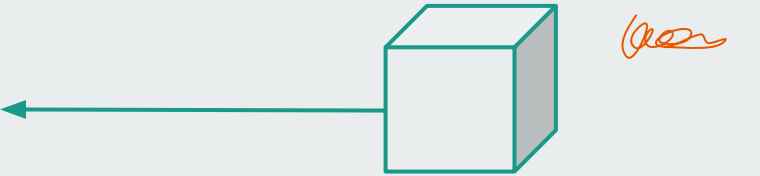


number of *for block B*
+ number of *for non leader blocks* **> f + p**

→ B unlocked

Unlocked Block - Case 1



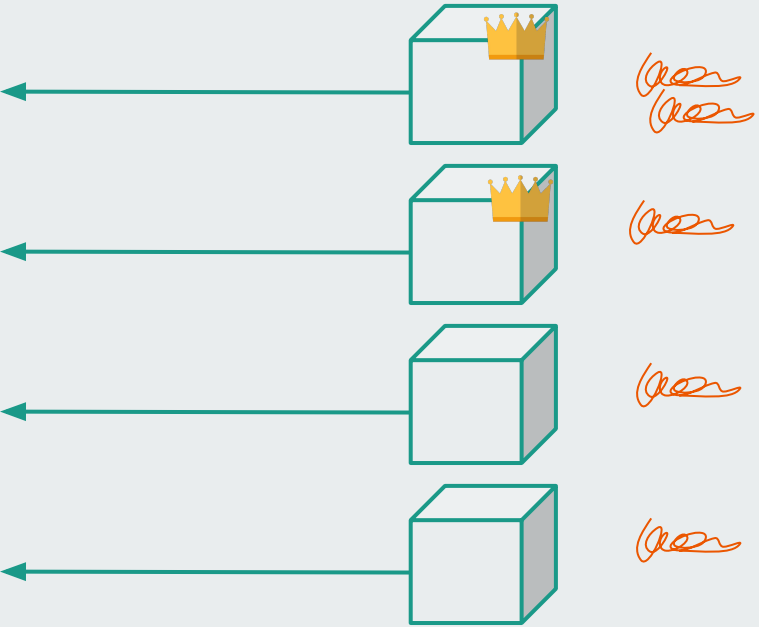


number of *for block B*
+ number of *for non leader blocks* ***> f + p***

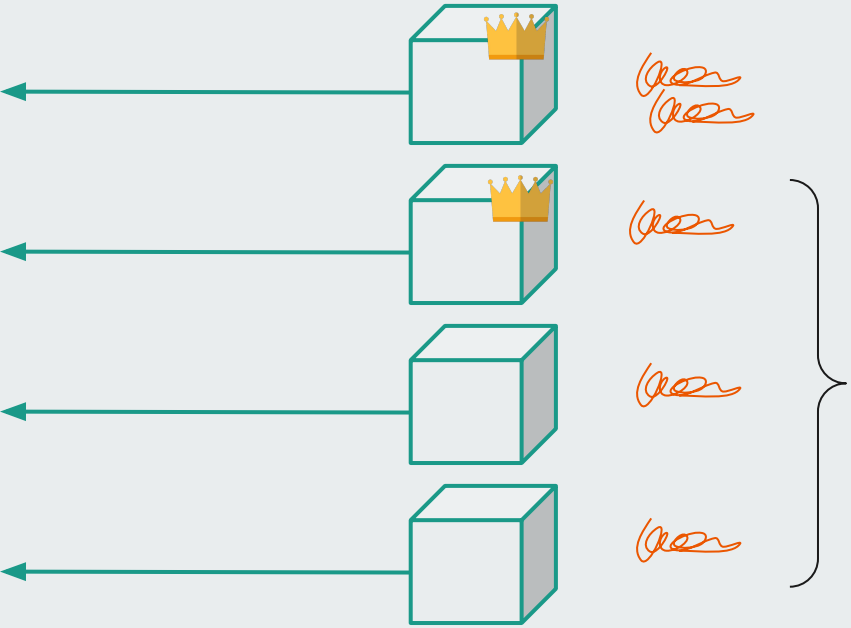
→ B unlocked

Can never be fast finalized.

Unlocked Block - Case 2



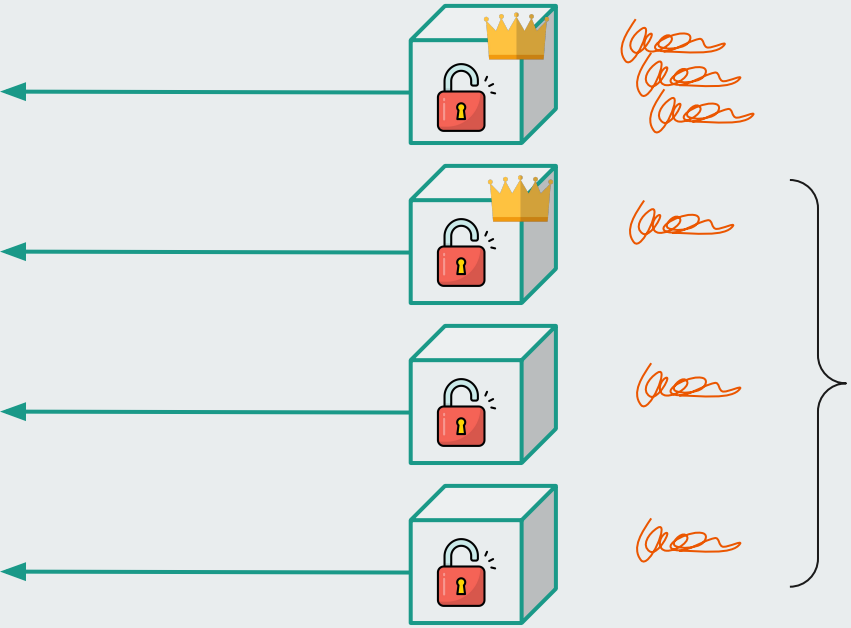
Unlocked Block - Case 2



number of *for non-max blocks* ***> f + p***

→ all unlocked

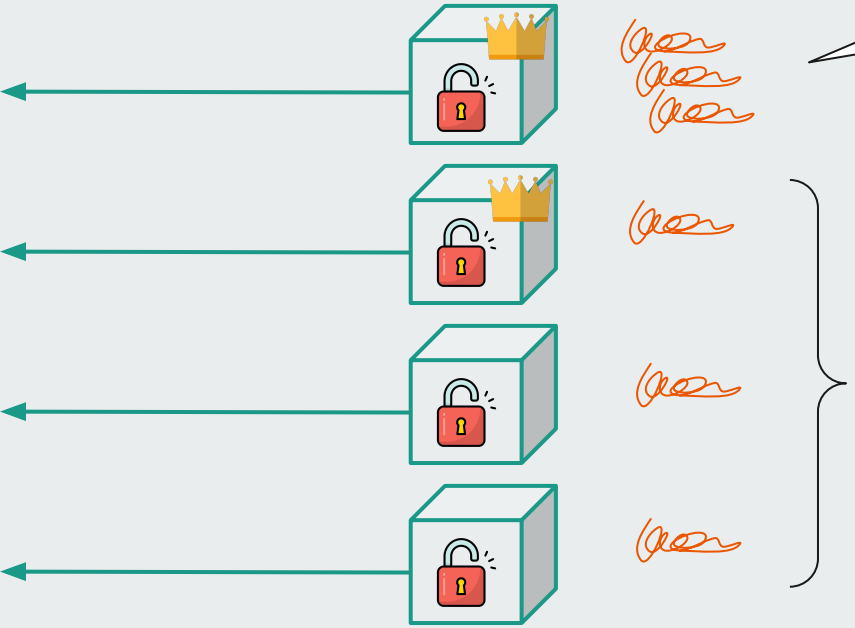
Unlocked Block - Case 2



number of *for non-max blocks* **> *f* + *p***

→ all unlocked

Unlocked Block - Case 2



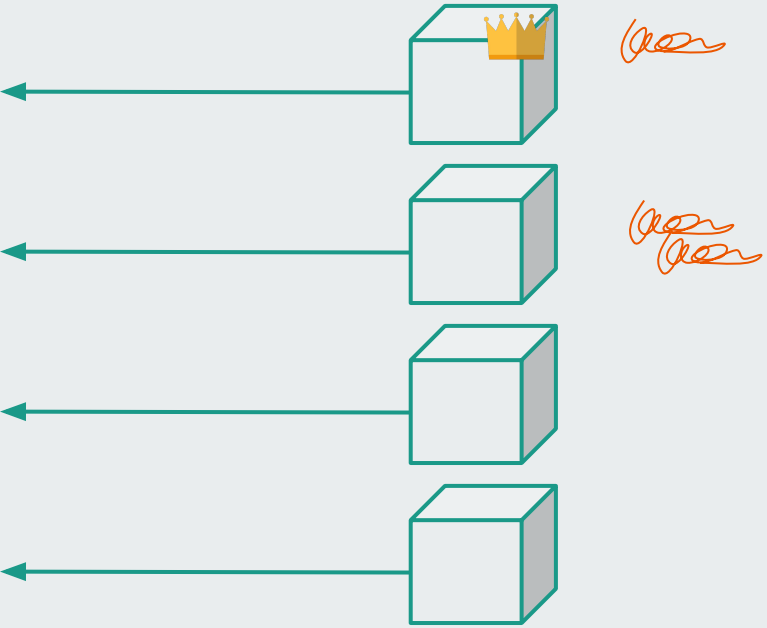
No block will be fast finalized.

number of *for non-max blocks* **> *f* + *p***

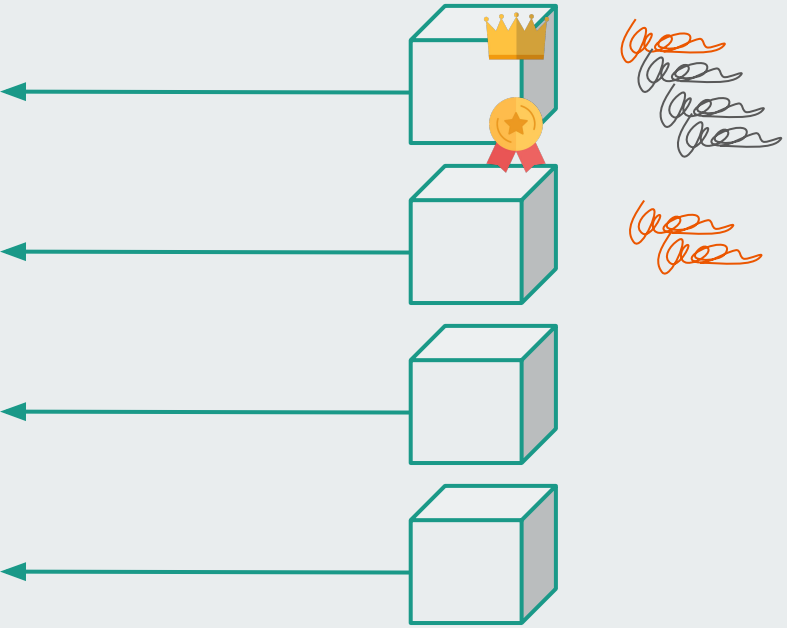
→ all unlocked

New Rule = Restriction ?

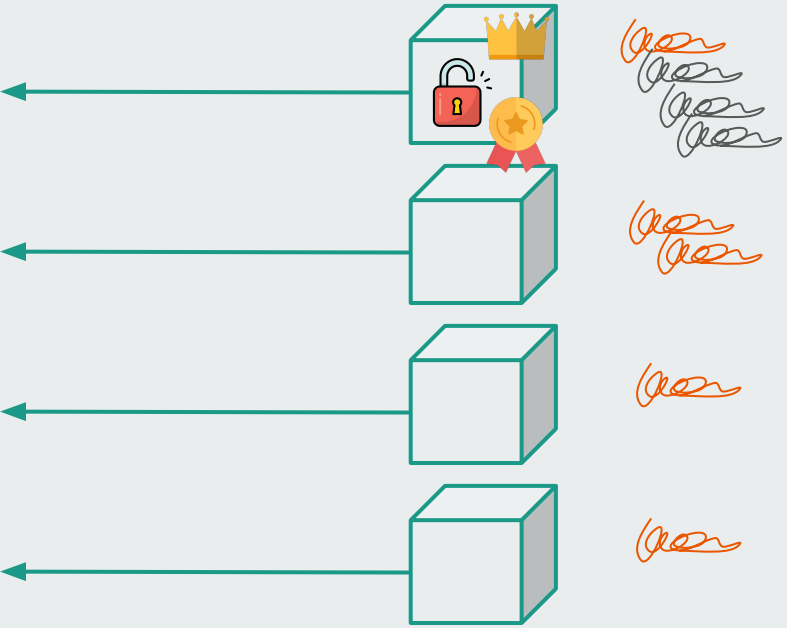
New Rule = Restriction ?



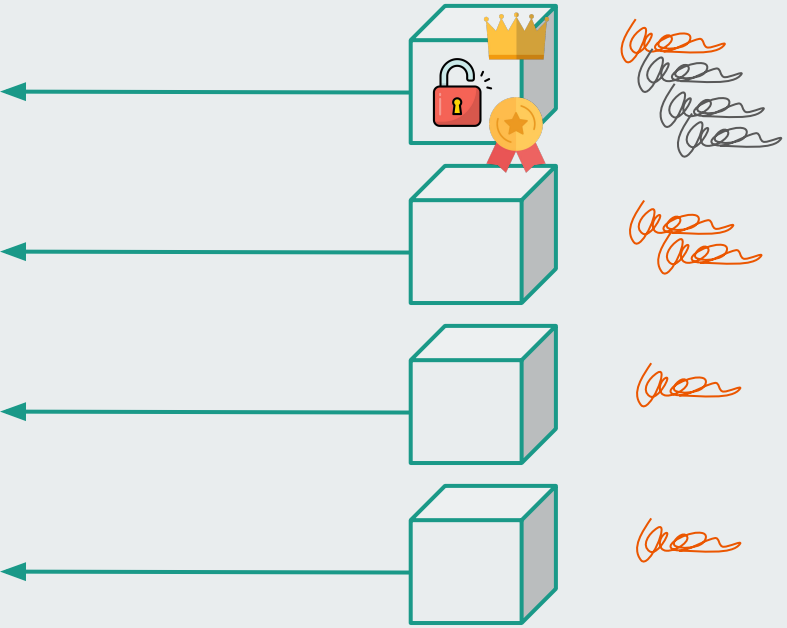
New Rule = Restriction ?



New Rule = Restriction ?



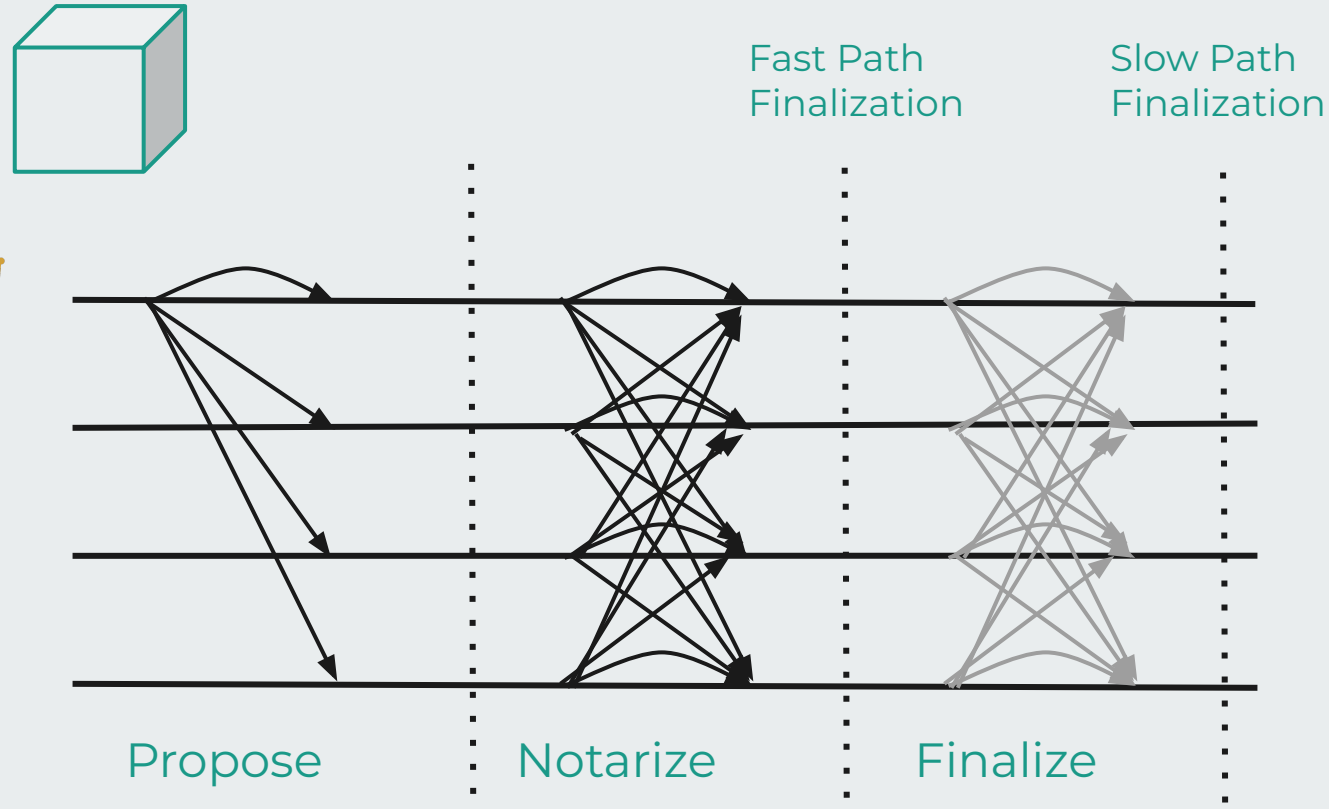
New Rule = Restriction ?



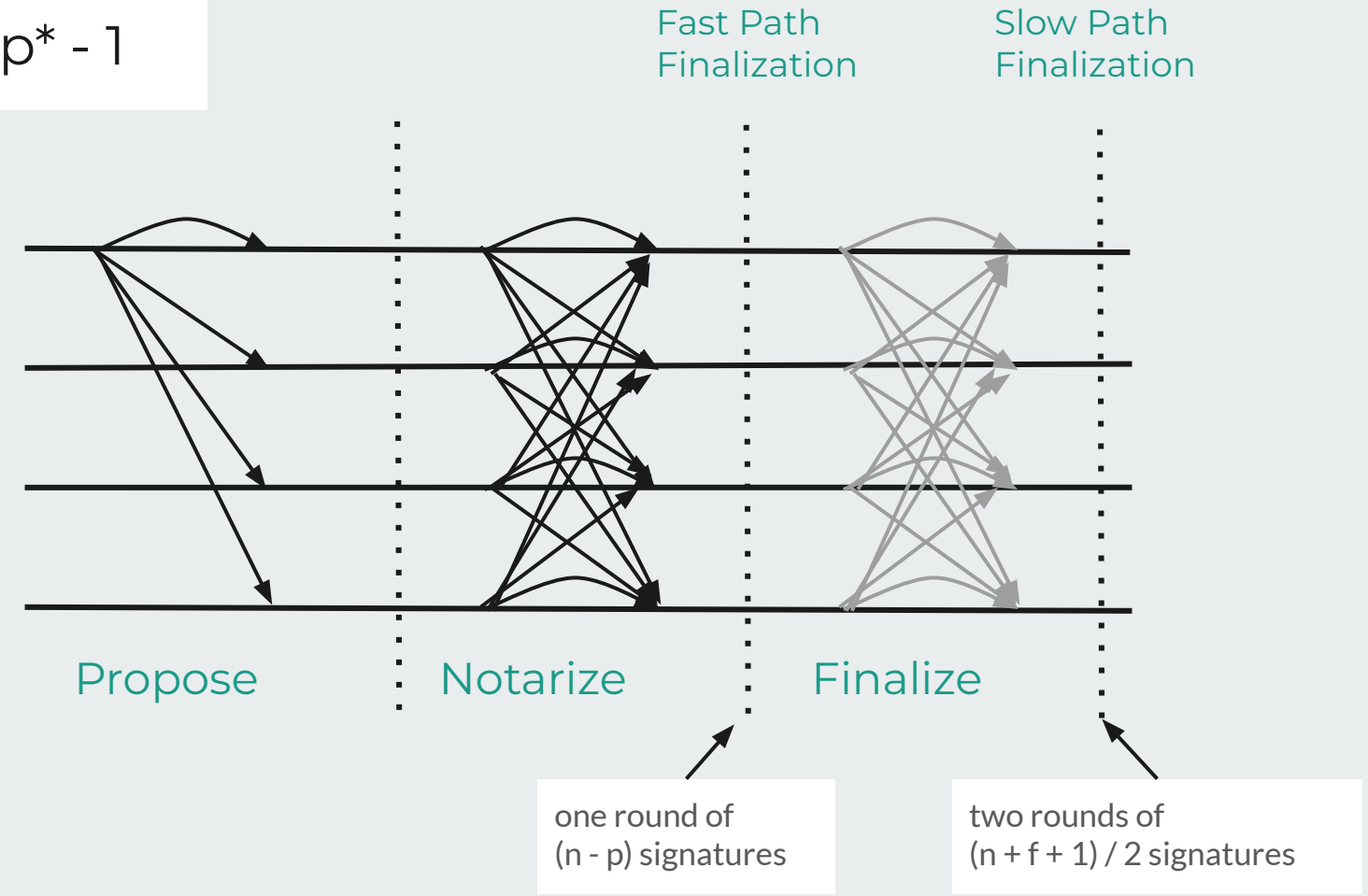
Worst case: waiting for

(n - f)

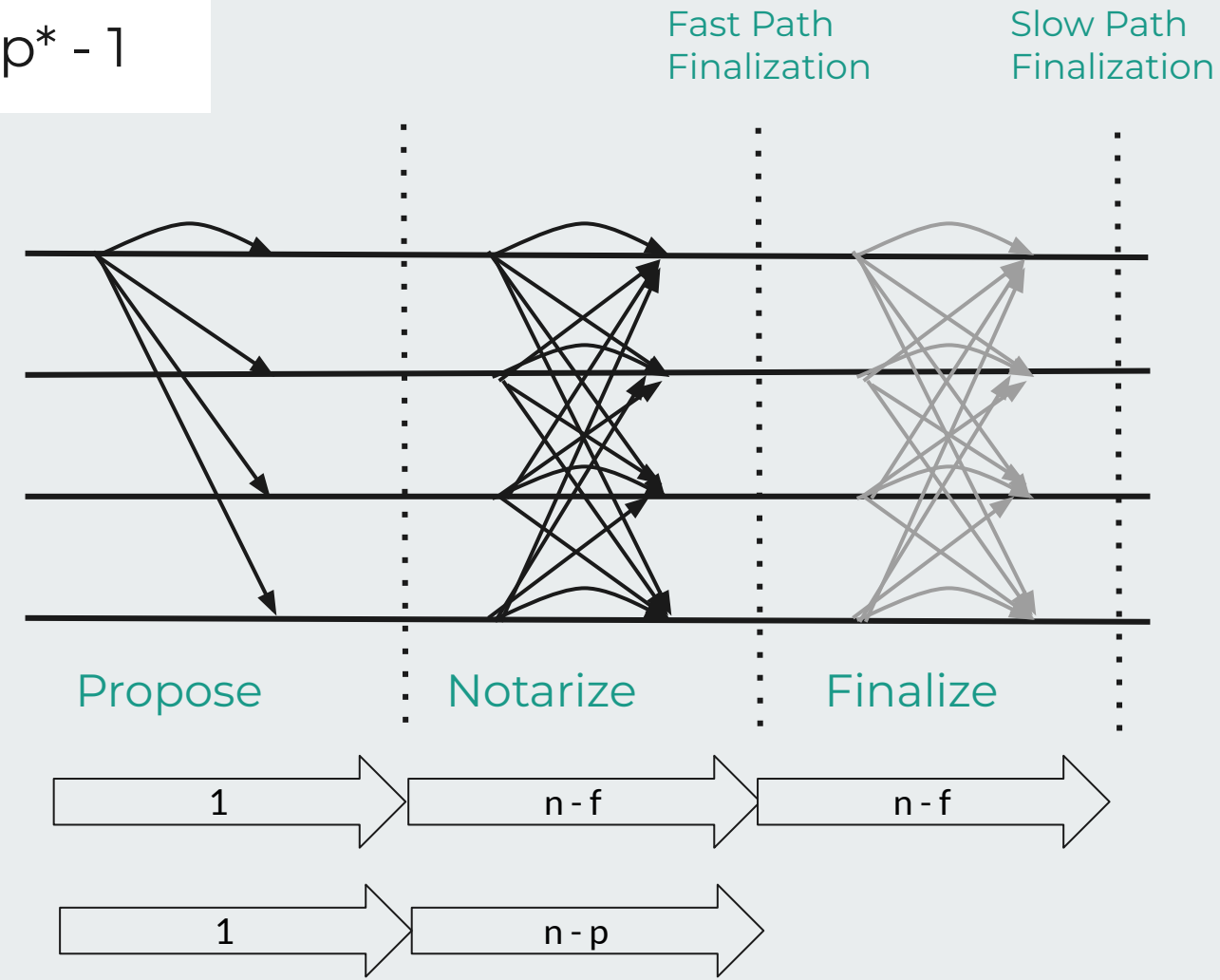
while a block is already notarized.


Fast Path
Finalization
Slow Path
Finalization
Propose
Notarize
Finalize

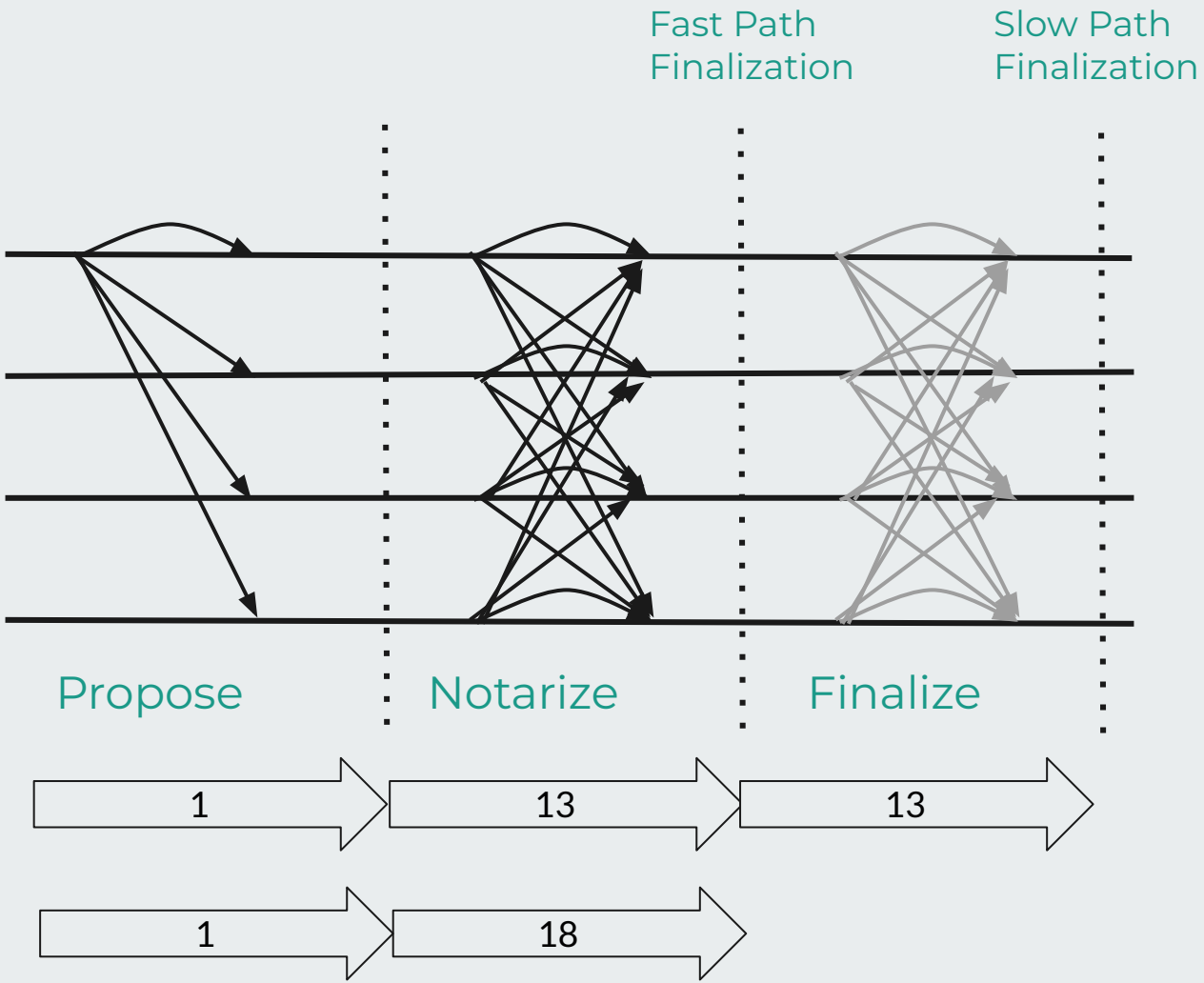
$$n \geq 3f + 2p^* - 1$$


Fast Path
Finalization
Slow Path
Finalization
Propose
Notarize
Finalize
one round of
(n - p) signatures
two rounds of
(n + f + 1) / 2 signatures

$$n \geq 3f + 2p^* - 1$$


Fast Path Finalization
Slow Path Finalization
Propose
Notarize
Finalize
1
n - f
n - f
1
n - p

f = 6
n = 19
p = 1


Fast Path Finalization
Slow Path Finalization
Propose
Notarize
Finalize
1
13
13
1
18

	Block Finalization Latency	Block Finalization Requirement	Block Creation Latency	Block Creation Requirement	Number of Replicas *equals*	Supports Rotating Leaders
Casper FFG [18]	$O(\Delta)$	$2f+1$	$O(\Delta)$	N/A[†]	$3f+1$	✓
Fast HotStuff [40][‡]	5δ	$2f+1$	2δ	$2f+1$	$3f+1$	
Jolteon [36]	5δ	$2f+1$	2δ	$2f+1$	$3f+1$	
PaLa [23]	4δ	$2f+1$	2δ	$2f+1$	$3f+1$	
Zelma [2]	2δ	$3f+p+1$	2δ	$2f+p+1$	$3f+2p+1$	
SBFT [39]	2δ	$3f+p+1$	3δ[§]	$2f+p+1$	$3f+2p+1$	
Streamlet [22]	6Δ	$2f+1$	2Δ	$2f+1$	$3f+1$	✓
Bullshark [60]	4δ[¶]	$2f+1$	2δ	$2f+1$	$3f+1$	✓
BBCA-Chain [51]	3δ	$2f+1$	3δ	$2f+1$	$3f+1$	✓
ICC [19] / Simplex [21]	3δ	$2f+1$	2δ	$2f+1$	$3f+1$	✓
Mysticeti [9]	3δ	$2f+1$	1δ	$2f+1$	$3f+1$	✓
Banyan	2δ	$3f+p^*-1$	2δ	$2f+p^*$	$3f+2p^*-1$	✓

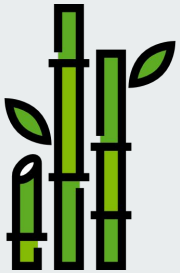
Related Work

	Block Finalization Latency	Block Finalization Requirement	Block Creation Latency	Block Creation Requirement	Number of Replicas *equals*	Supports Rotating Leaders
Casper FFG [18]	$O(\Delta)$	$2f+1$	$O(\Delta)$	N/A[†]	$3f+1$	✓
Fast HotStuff [40][‡]	5δ	$2f+1$	2δ	$2f+1$	$3f+1$	
Jolteon [36]	5δ	$2f+1$	2δ	$2f+1$	$3f+1$	
PaLa [23]	4δ	$2f+1$	2δ	$2f+1$	$3f+1$	
Zelma [2]	2δ	$3f+p+1$	2δ	$2f+p+1$	$3f+2p+1$	
SBFT [39]	2δ	$3f+p+1$	3δ[§]	$2f+p+1$	$3f+2p+1$	
Streamlet [22]	6Δ	$2f+1$	2Δ	$2f+1$	$3f+1$	✓
Bullshark [60]	4δ[¶]	$2f+1$	2δ	$2f+1$	$3f+1$	✓
BBCA-Chain [51]	3δ	$2f+1$	3δ	$2f+1$	$3f+1$	✓
ICC [19] / Simplex [21]	3δ	$2f+1$	2δ	$2f+1$	$3f+1$	✓
Mysticeti [9]	3δ	$2f+1$	1δ	$2f+1$	$3f+1$	✓
Banyan	2δ	$3f+p^*-1$	2δ	$2f+p^*$	$3f+2p^*-1$	✓

Related Work

Evaluation

Bamboo BFT



Provides reference **HotStuff** and **Streamlet** implementations.

We add **ICC** and **Banyan**.

Dissecting the Performance of Chained-BFT

Fangyu Gai*, Ali Farahbakhsh*, Jianyu Niu*, Chen Feng*, Ivan Beschastnikh†, Hao Duan‡
University of British Columbia (*Okanagan Campus, †Vancouver Campus)
‡Hangzhou Qulian Technology Co., Ltd.

arXiv:2103.00777v1 [cs.CR] 1 Mar 2021

***Abstract*—Permissioned blockchains employ Byzantine fault-tolerant (BFT) state machine replication (SMR) to reach agreement on an ever-growing, linearly ordered log of transactions. A new paradigm, combined with decades of research in BFT SMR and blockchain (namely chained-BFT, or cBFT), has emerged for directly constructing blockchain protocols. Chained-BFT protocols have a unifying propose-vote scheme instead of multiple different voting phases with a set of voting and commit rules to guarantee safety and liveness. However, distinct voting and commit rules impose varying impacts on performance under different workloads, network conditions, and Byzantine attacks. Therefore, a fair comparison of the proposed protocols poses a challenge that has not yet been addressed by existing work.**

We fill this gap by studying a family of cBFT protocols with a two-pronged systematic approach. First, we present an evaluation framework, Bamboo, for quick prototyping of cBFT protocols and that includes helpful benchmarking facilities. To validate Bamboo, we introduce an analytic model using queuing theory which also offers a back-of-the-envelope guide for dissecting these protocols. We build multiple cBFT protocols using Bamboo and we are the first to fairly compare three representatives (i.e., HotStuff, two-chain HotStuff, and Streamlet). We evaluated these protocols under various parameters and scenarios, including two Byzantine attacks that have not been widely discussed in the literature. Our findings reveal interesting trade-offs (e.g., responsiveness vs. forking-resilience) between different cBFT protocols and their design choices, which provide developers and researchers with insights into the design and implementation of this protocol family.

***Index Terms*—Byzantine fault-tolerant, evaluation, performance, framework**

I. Introduction

Classic Byzantine fault-tolerant (BFT) state machine replication (SMR) protocols like PBFT [1] rely on a stable leader to drive the protocol until a view change occurs. This limits their scalability and deployability in a context that involves thousands of nodes and requires highly distributed trust. Recent work has been exploring an alternative, called *chained-BFT* or cBFT, combining decades of research in BFT and state-of-the-art blockchain work, which is considered to be the next generation BFT for blockchains.

The run-time of a cBFT protocol is divided into views. Each view has a designated leader, which is elected via some leader election protocol. Unlike classical BFT protocols, such as PBFT [1], in which each proposed block has to go through three different phases of voting, cBFT protocols use the chained structure that enables a single-phase *Propose-Vote* scheme. Since blocks are cryptographically linked together, a vote cast on a block is also cast on the preceding blocks of the same branch. This allows for streamlining of decisions and fewer messages types, and also eases the burden of reasoning

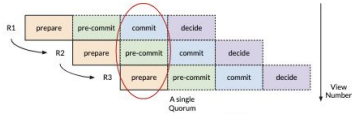


Fig. 1: Run-time of chained-HotStuff [2]. Replicas R_1, R_2, and R_3 are rotating to propose blocks. A QC accumulated at a phase for a single block can serve in different phases of the preceding blocks.

about the correctness of the protocols. See an illustration of chained HotStuff in Figure 1. Votes for the same block from distinct replicas are accumulated as a *Quorum Certificate* (short for QC) if a threshold is reached (i.e., over two thirds of all the nodes) and the QC is also recorded on the blockchain along with the relevant block for bookkeeping.

Liveness and safety properties are guaranteed by voting and commit rules, both of which characterize a cBFT protocol. Informally, a *voting rule* requires that an incoming block is voted on only if it extends a valid branch, while a *commit rule* requires that a chain of blocks is committed by removing a certain number of blocks at the tail of the blockchain. However, the choice of rules changes performance under different conditions: network environment (latency and bandwidth), workload, deployment parameters, and failures.

The idea of connecting modern blockchains with classic BFT originates implicitly in Casper-FFG [3], followed by a series of recent proposals [2]–[8]. Many companies have been using these protocols. For example, Diem from Facebook (previously known as Libra), Flow from Dapper Labs, and Hyperchain from Qulian have deployed cBFT protocols as their core infrastructure for payment, gaming, and supply chain systems [9]–[11]. Understanding the performance of these protocols is important for the overall distributed system. However, these protocols differ in their voting and commit rules and design choices. To our knowledge, there has been no study that empirically provides a fair and comprehensive comparison of these protocols.

To dissect and explain the performance of chained-BFT variants, we propose a novel framework called *Bamboo*. Bamboo provides well-defined interfaces so that developers can quickly prototype their own cBFT protocols by defining voting/commit rules. We used Bamboo to build multiple

Methodology

- AWS t3.large
  - 2 virtual cores
  - 8GB of RAM
- Rank delay high enough for the rank 0 block to be propagated everywhere
- Use **proposer latency** as metric
  - proposer latency = (finalization time at proposer) – (time of block broadcast)





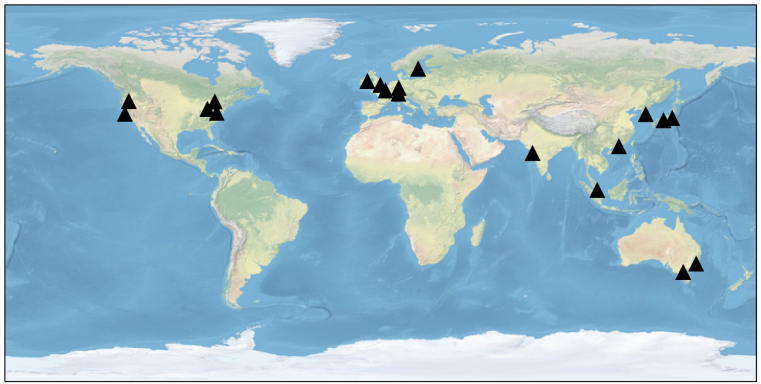
n = 19 f = 6



AWS t3.large



proposer latency





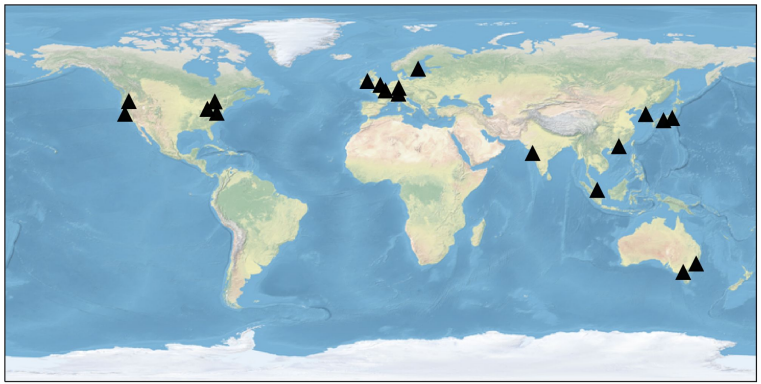
n = 19 f = 6

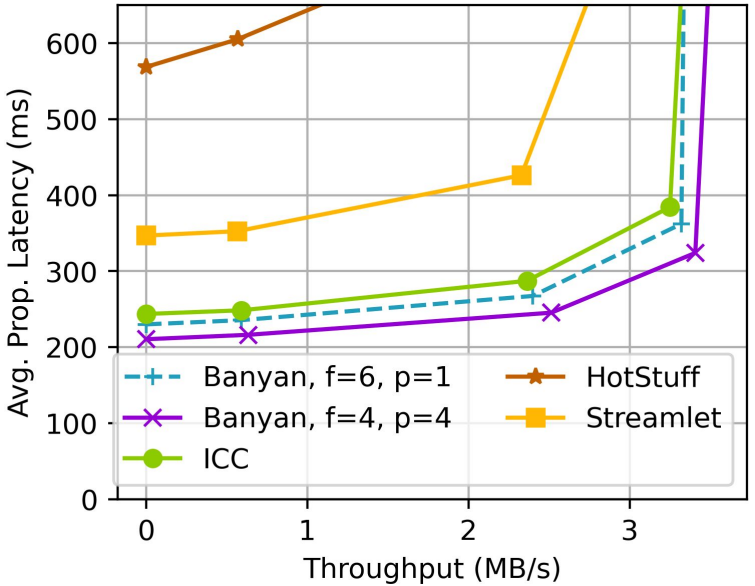


AWS t3.large



proposer latency



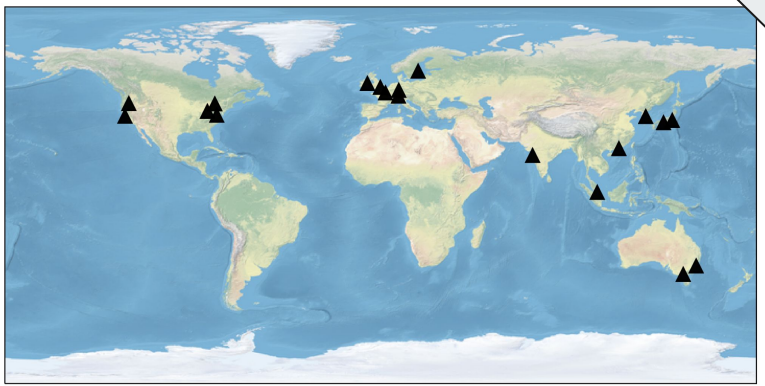


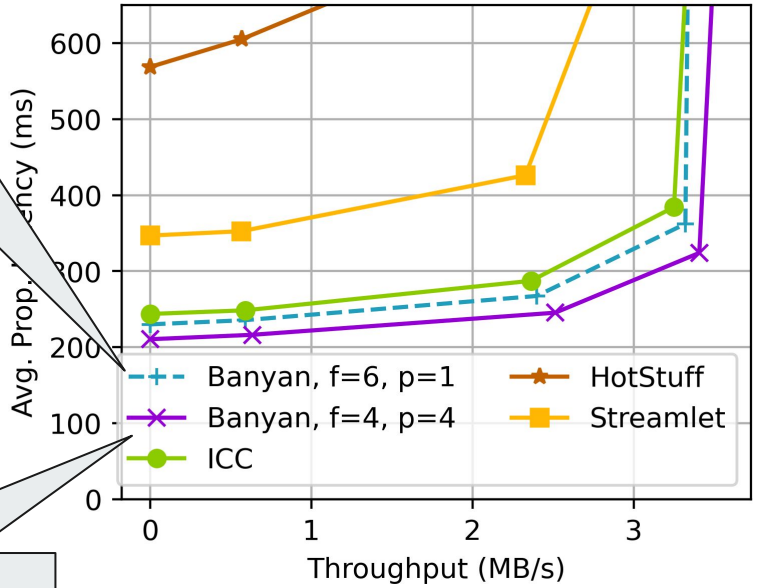
n = 19 f = 6

AWS t3.large

proposer latency

5.81% improvement



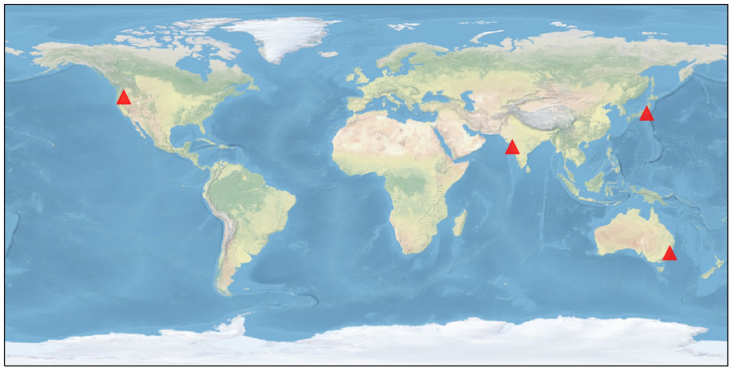


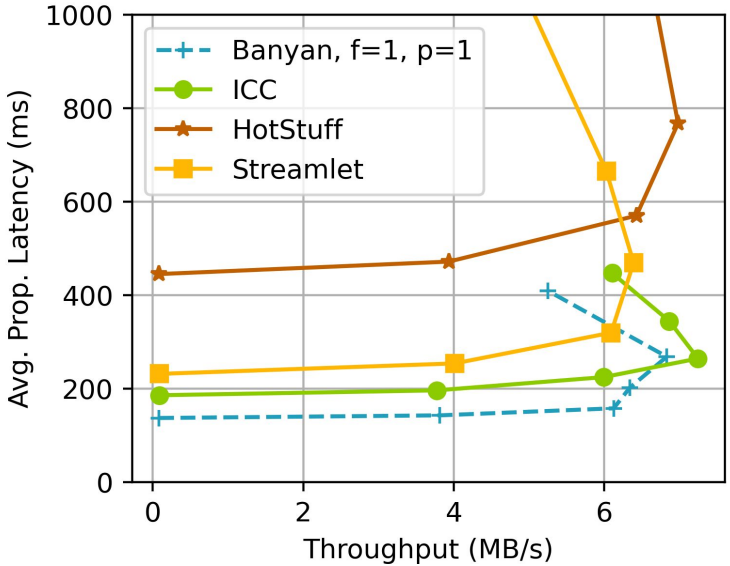
16.0% improvement

n = 19 f = 6

AWS t3.large

proposer latency

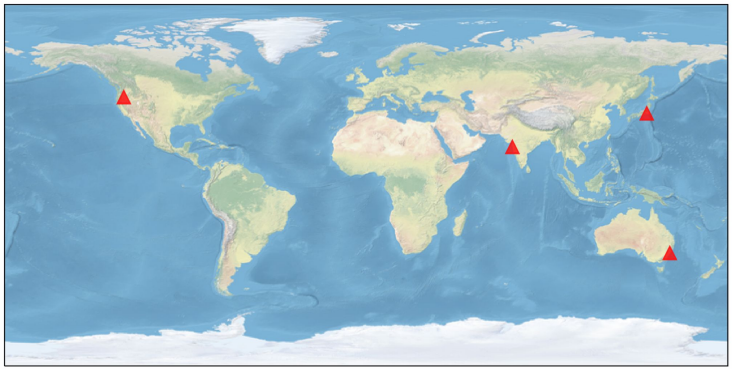


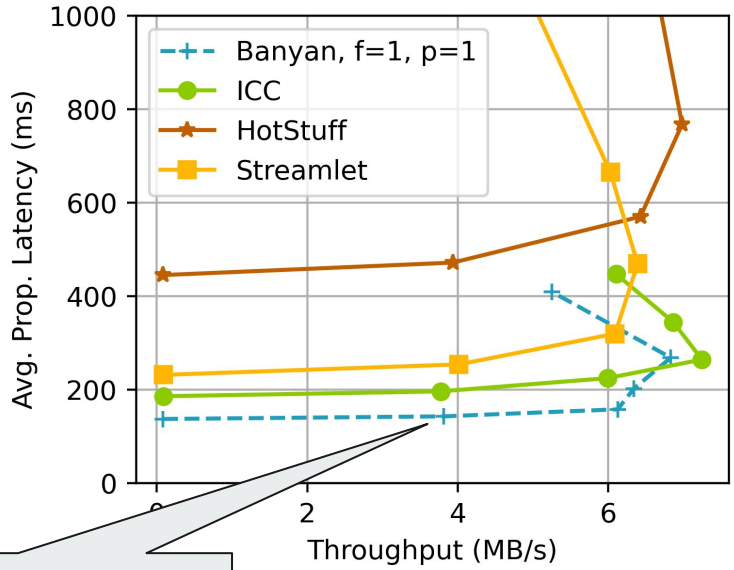


n = 4 f = 1

AWS t3.large

proposer latency



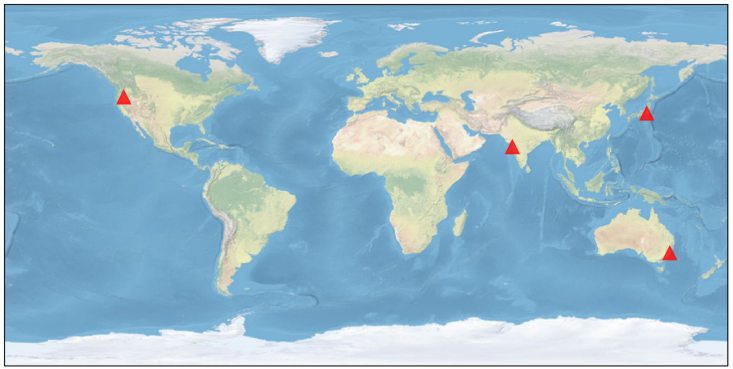


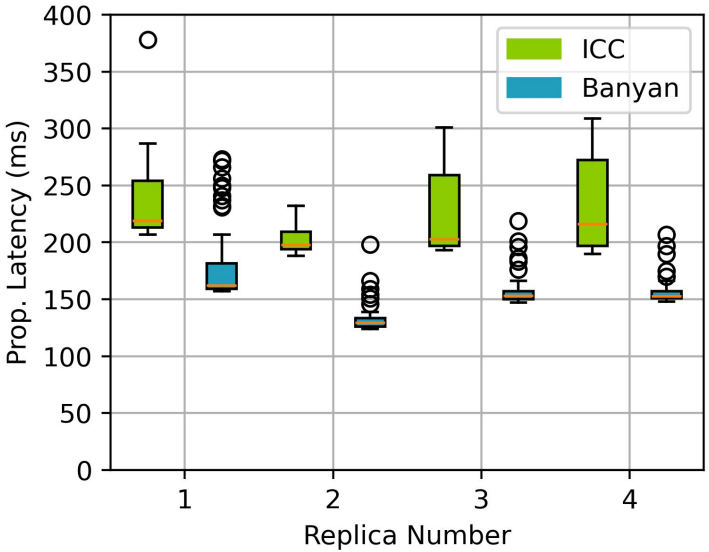
29.9% improvement

n = 4 f = 1

AWS t3.large

proposer latency

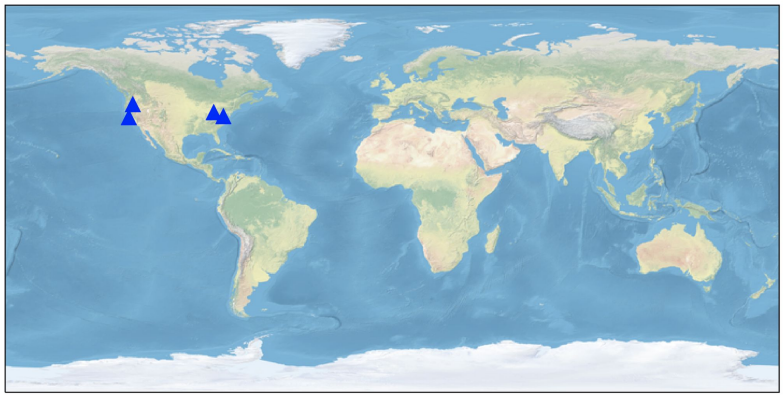


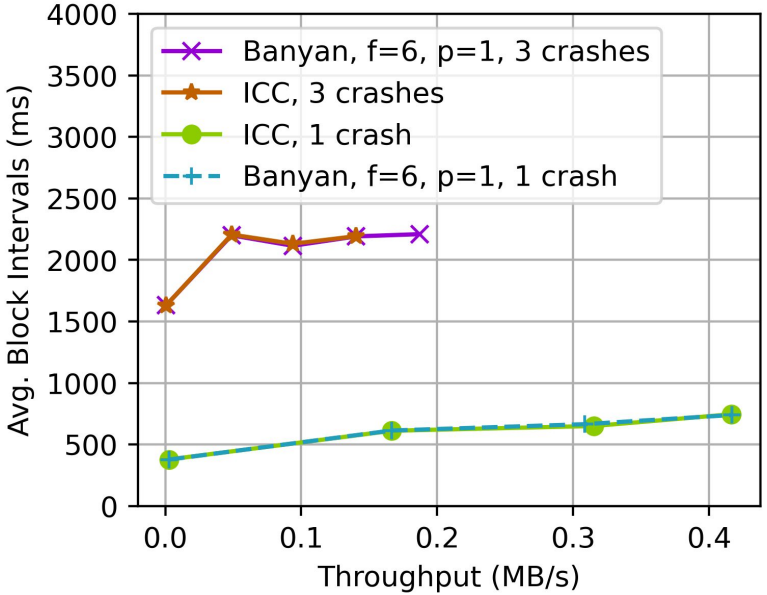


n = 4 f = 1

AWS t3.large

proposer latency







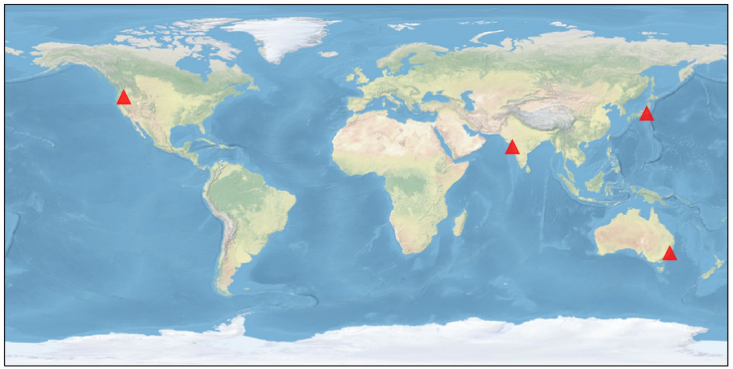
n = 19 f = 6

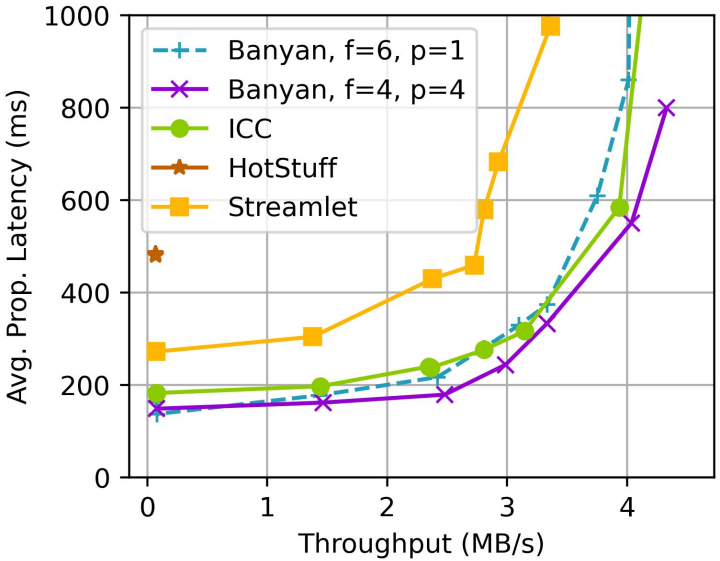


AWS t3.large

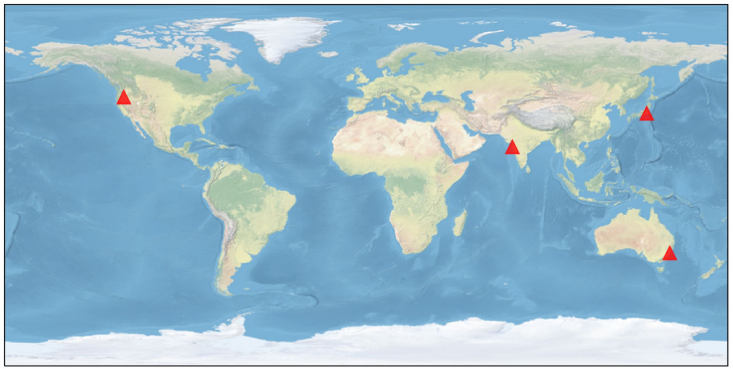


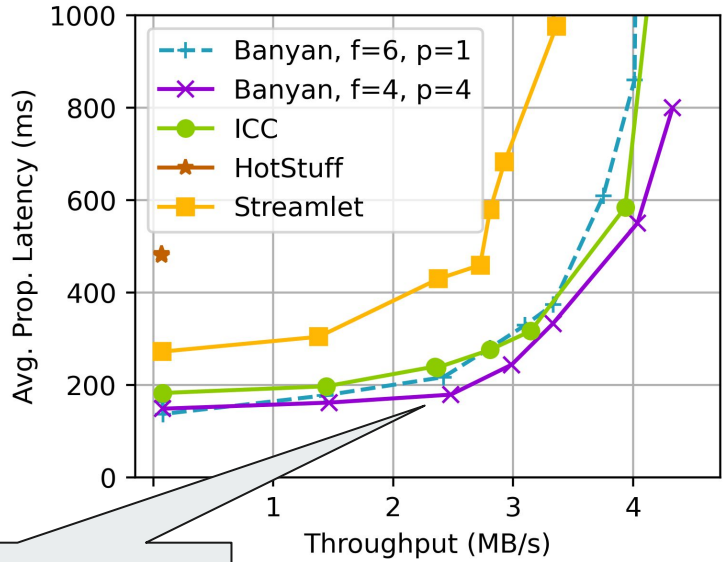
proposer latency





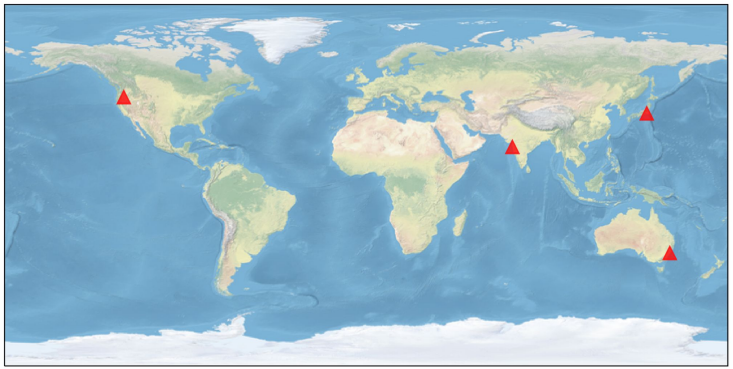
Throughput vs. proposal latency for **n=19** replicas spread across 4 global datacenters.

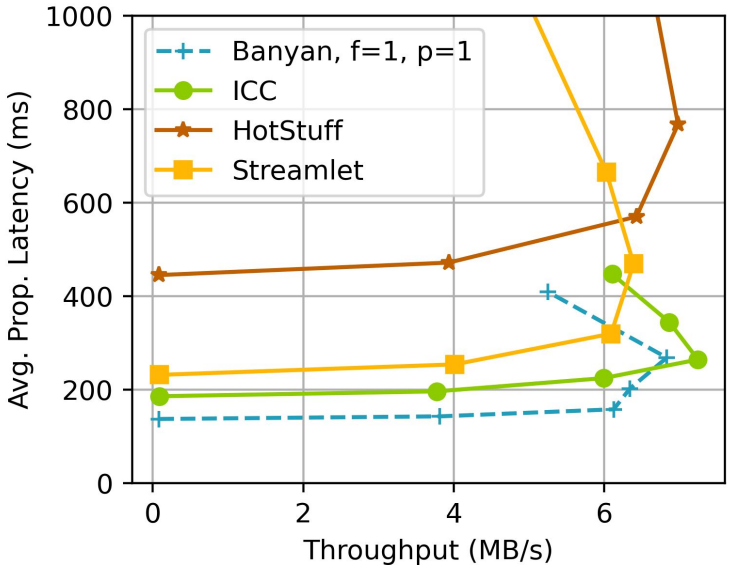




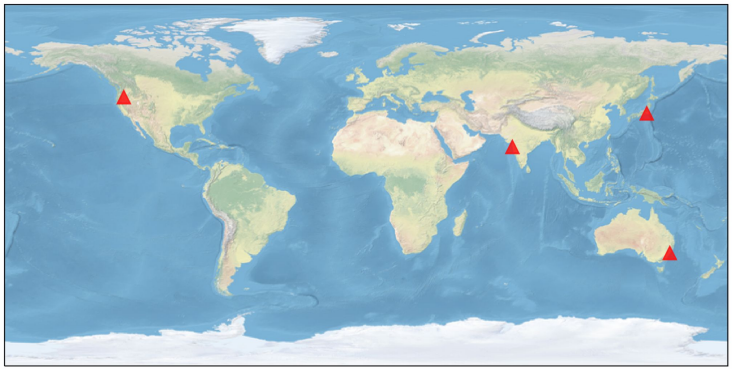
25.1% improvement

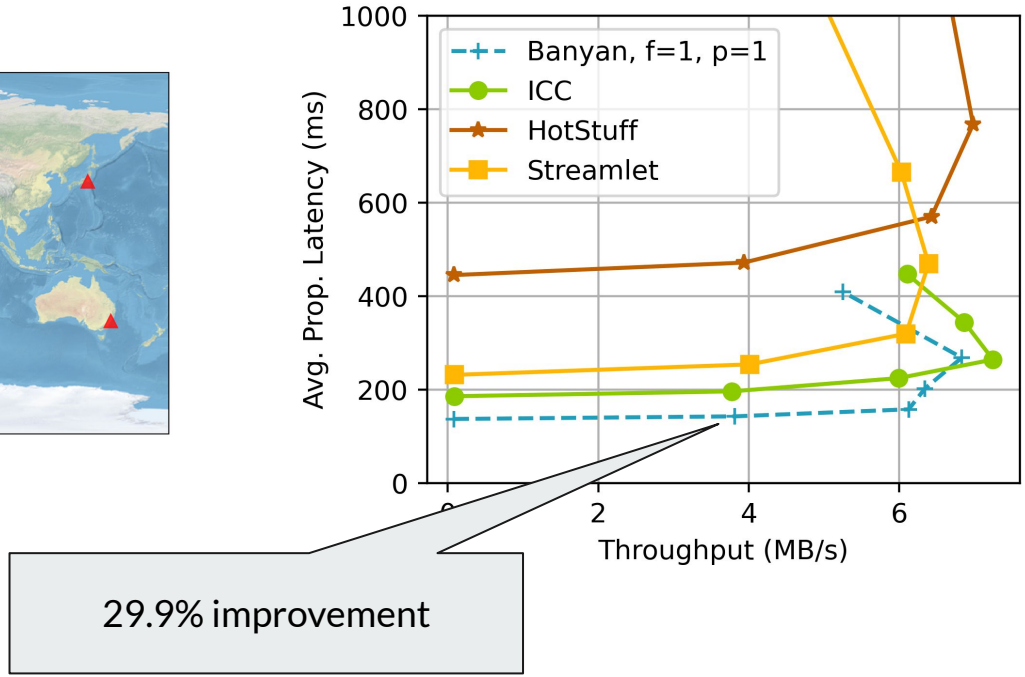
Throughput vs. proposal latency for **n=19** replicas spread across 4 global datacenters.



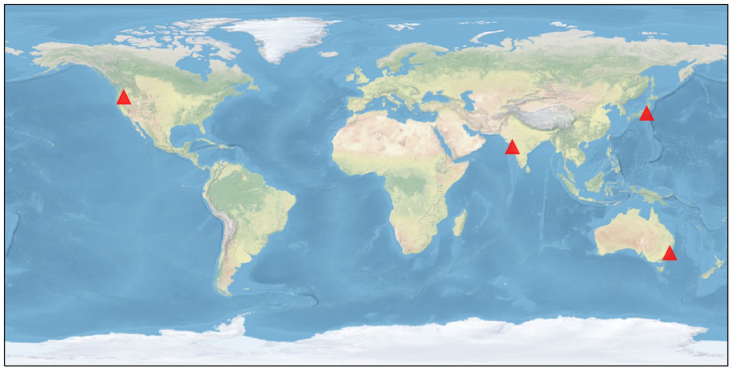


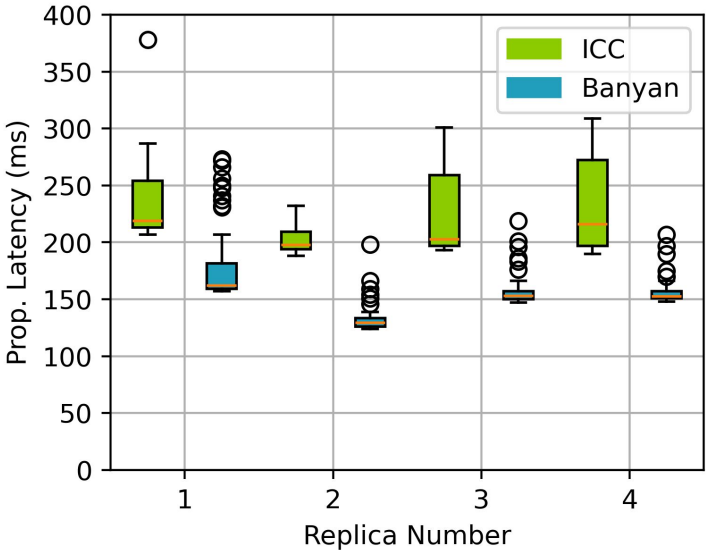
Throughput vs. proposal latency for **n=4** replicas spread across 4 global datacenters.



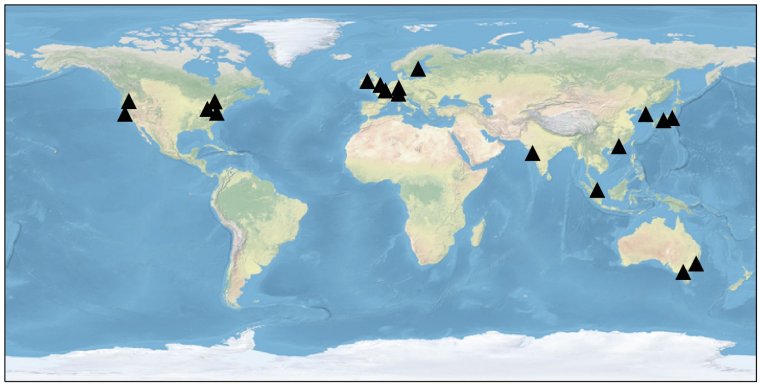


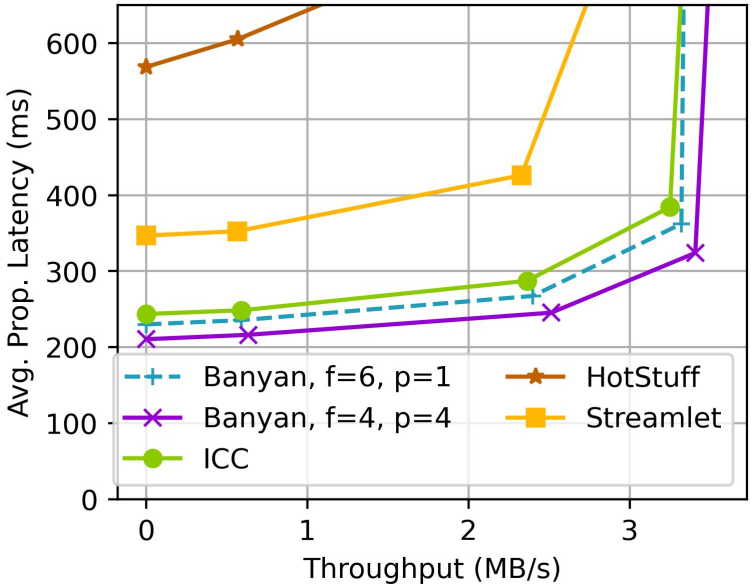
Throughput vs. proposal latency for **n=4** replicas spread across 4 global datacenters.



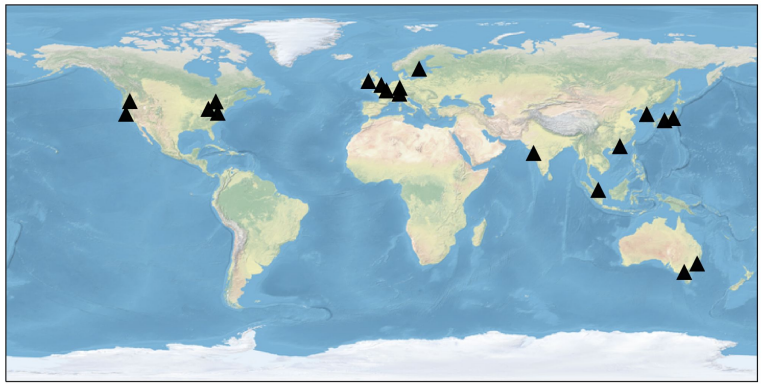


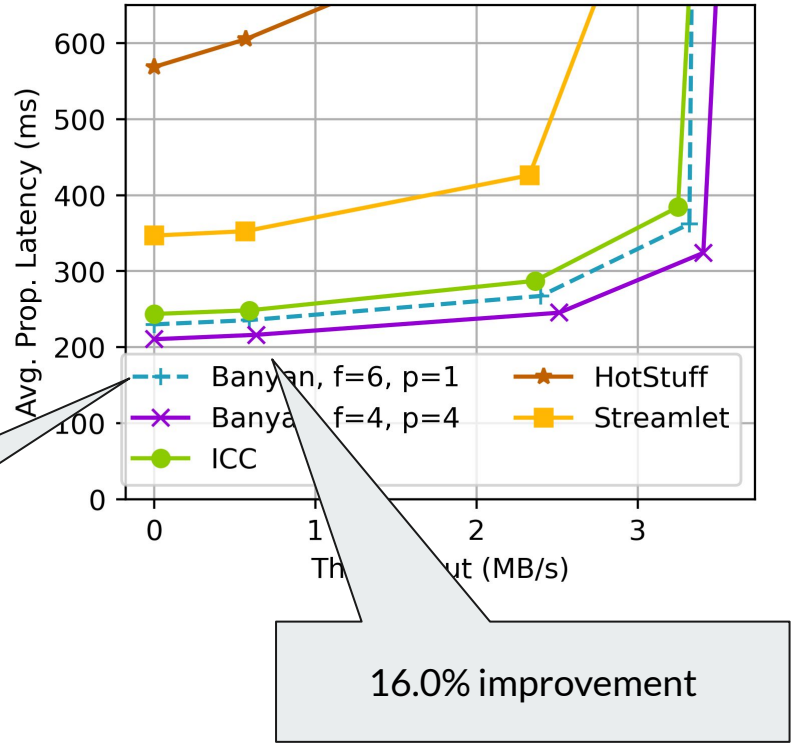
Variance of Banyan and ICC proposal latencies compared with **1 MB** payload.



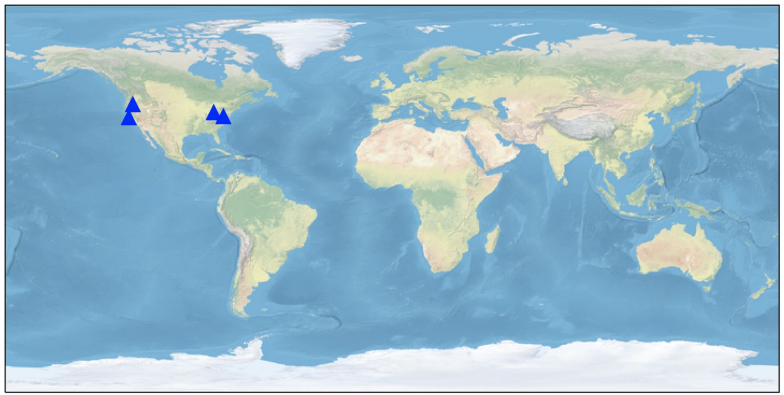


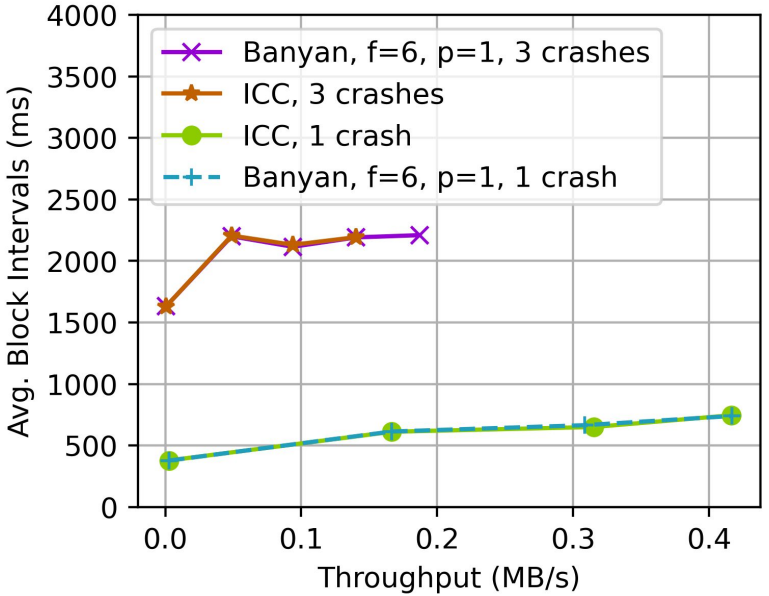
Throughput vs. proposal latency for **n=19** replicas spread across a global network.





Throughput vs. proposal latency for **n=19** replicas spread across a global network.





Effect of **crash-faults** on throughput and block intervals for n=19 replicas, spread across 4 US datacenters.

Main Takeaways

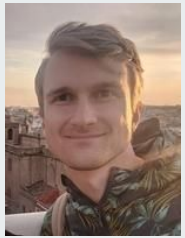
1. Banyan is faster than state-of-the-art (*optimistically*)



2. Banyan is never slower than ICC



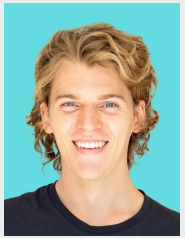




Yann Vonlanthen
ETH Zurich



Dr. Jakub Sliwinski
ETH Zurich

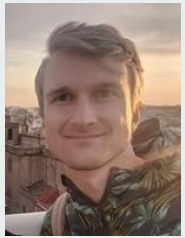


Massimo Albarello
ETH Zurich



Prof. Roger Wattenhofer
ETH Zurich

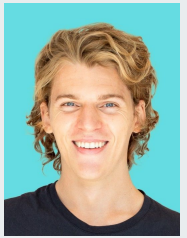




Yann Vonlanthen
ETH Zurich



Dr. Jakub Sliwinski
ETH Zurich



Massimo Albarello
ETH Zurich



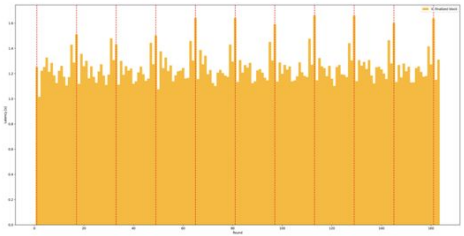
Prof. Roger Wattenhofer
ETH Zurich

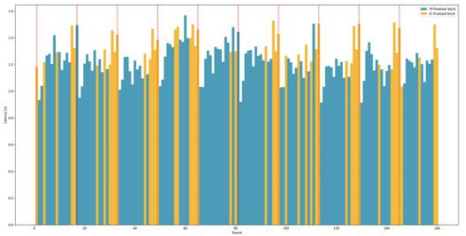
Desired Properties

- **Deadlock Freeness**
  - In each round, the block-tree height increases by 1.
- **Safety**
  - If some honest replica finalizes block b in round k, and another honest replica finalizes block b' in round k, then $b = b'$.
- **Liveness**
  - If the network is momentarily synchronous and the leader is honest, then the block proposed by the leader is added to the block-tree and finalized.
- **Fast Termination**
  - If the network is momentarily synchronous, the leader is honest, and $n-p$ replicas behave momentarily honestly, then the block proposed by the leader is added to the block-tree and finalized in a single round trip time.

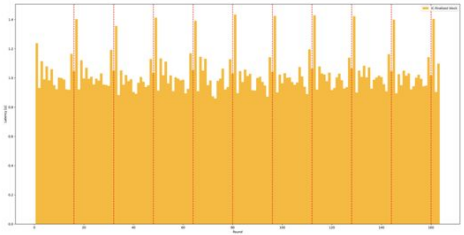


Sydney



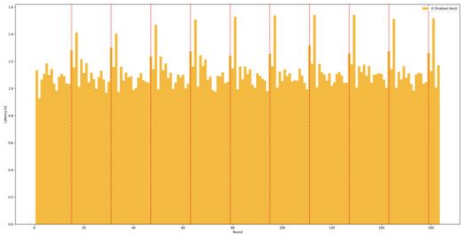


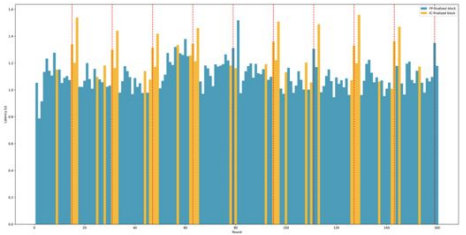
Seoul





Singapore





Very simple, but maybe not as simple as it looks.

[Submitted on 4 Dec 2017]

Revisiting Fast Practical Byzantine Fault Tolerance

Ittai Abraham, Guy Gueta, Dahlia Malkhi, Lorenzo Alvisi, Rama Kotla, Jean-Philippe Martin

In this note, we observe a safety violation in Zyzzyva and a liveness violation in FaB. To demonstrate these issues, we require relatively simple scenarios, involving only four replicas, and one or two view changes. In all of them, the problem is manifested already in the first log slot.

Subjects: **Distributed, Parallel, and Cluster Computing (cs.DC)**
Cite as: arXiv:1712.01367 **[cs.DC]**
(or arXiv:1712.01367v1 **[cs.DC]** for this version)
https://doi.org/10.48550/arXiv.1712.01367